



Labs de GitLab Básico

Autor:: Álvaro Bergantiños

Fecha: : 03/2026

Contenidos

1. Lab: Instalación y configuración de Git	1
1.1. Objetivos	1
1.2. Prerrequisitos	1
1.3. Ejercicios	1
1.3.1. Comprobación inicial del entorno	1
1.3.2. Instalación de Git en Fedora	1
1.3.3. Configuración inicial global	2
1.3.4. Alias de uso frecuente	2
1.3.5. Verificación de configuración y origen	2
1.3.6. Preparación de un .gitignore base de práctica	2
1.4. Resultado esperado	3
2. Lab: Ciclo de vida de los ficheros y sistema interno de Git	4
2.1. Objetivos	4
2.2. Prerrequisitos	4
2.3. Ejercicios	4
2.3.1. Retomar el proyecto del laboratorio anterior	4
2.3.2. Crear un fichero y observar el estado untracked	4
2.3.3. Ver dónde está el index y qué contenido tiene	5
2.3.4. Añadir al staging area y verificar el index	5
2.3.5. Crear un nuevo commit e inspeccionar commit y tree	5
2.3.6. Modificar el fichero para ver la transición modified → staged	6
2.3.7. Navegar por ambos estados	7
2.4. Resultado esperado	7
3. Lab: Comandos básicos de trabajo local con Git	8
3.1. Objetivos	8
3.2. Prerrequisitos	8
3.3. Ejercicios	8
3.3.1. Retomar el proyecto formativo	8
3.3.2. Preparar cambios para inspección	8
3.3.3. Comparar working tree, staging area y último commit	9
3.3.4. Leer historial con distintos niveles de detalle	9
3.3.5. Inspeccionar un commit concreto con git show	10
3.3.6. Navegar por el historial con referencias relativas	10
3.3.7. Usar git blame para rastrear el origen de una línea	10
3.3.8. Crear un clon local para auditoría rápida	11
3.4. Resultado esperado	11
4. Lab: Stash para guardar trabajo inacabado	12
4.1. Objetivos	12
4.2. Prerrequisitos	12

4.3. Ejercicios	12
4.3.1. Preparar el escenario de interrupción	12
4.3.2. Riesgos de cambiar de rama con trabajo inacabado	12
4.3.3. Crear trabajo en curso sin terminar	14
4.3.4. Guardar el trabajo temporal en un stash con mensaje	14
4.3.5. Resolver la interrupción urgente	15
4.3.6. Recuperar los cambios guardados	15
4.3.7. Guardar también archivos no rastreados	15
4.3.8. Separar cambios preparados y no preparados con keep-index	16
4.3.9. git worktree como alternativa al stash	16
4.4. Resultado esperado	18
5. Lab: Comandos avanzados de gestión del historial	20
5.1. Objetivos	20
5.2. Prerrequisitos	20
5.3. Ejercicios	20
5.3.1. Verificar el punto de partida	20
5.3.2. Corregir el último commit con amend	20
5.3.3. Corregir un commit que incluyó un fichero indebido	21
5.3.4. Limpiar artefactos no rastreados con seguridad	22
5.3.5. Revertir un cambio sin reescribir historial	22
5.3.6. Deshacer commits locales con reset	22
5.3.7. Recuperar commits perdidos con reflog	24
5.3.8. Reorganizar commits locales con rebase interactivo	25
5.3.9. Trasladar una corrección puntual con cherry-pick	26
5.4. Resultado esperado	26
6. Lab: Flujo feature branch con conflicto y resolución	28
6.1. Objetivos	28
6.2. Prerrequisitos	28
6.3. Ejercicios	28
6.3.1. Verificar el punto de partida	28
6.3.2. Merge fast-forward: rama sin conflicto	28
6.3.3. Preparar un conflicto entre ramas	29
6.3.4. Provocar y resolver el conflicto	30
6.3.5. Merge --no-ff explícito	31
6.3.6. Conflicto por fichero eliminado en una rama y modificado en otra	31
6.3.7. Abandonar un merge conflictivo	32
6.3.8. Resolver un conflicto aceptando íntegramente una versión	33
6.3.9. Merge con squash: condensar commits de una rama en uno solo	33
6.3.10. Limpiar ramas integradas	35
6.4. Resultado esperado	35
7. Lab: Etiquetado de release con versionado semántico	36

7.1. Objetivos	36
7.2. Ejercicios	36
7.2.1. Verificar el punto de partida	36
7.2.2. Crear una tag ligera	36
7.2.3. Crear una tag anotada para la release oficial	36
7.2.4. Listar y filtrar tags	37
7.2.5. Etiquetar un commit anterior	37
7.2.6. Limpiar la tag de release candidate	38
7.2.7. Publicación de tags al remoto (conceptual)	38
7.3. Resultado esperado	38
8. Lab: Configuración SSH y primer push a GitLab	39
8.1. Objetivos	39
8.2. Prerrequisitos	39
8.3. Ejercicios	39
8.3.1. Verificar el punto de partida	39
8.3.2. Generar el par de claves SSH	39
8.3.3. Registrar la clave pública en GitLab	40
8.3.4. Verificar la conexión SSH con GitLab	40
8.3.5. Crear un nuevo proyecto en GitLab	41
8.3.6. Configurar el remoto y hacer el primer push	41
8.3.7. Crear un Personal Access Token (ejercicio complementario)	41
8.4. Resultado esperado	42
9. Lab: Colaboración simulada con fetch, pull y push	43
9.1. Objetivos	43
9.2. Ejercicios	43
9.2.1. Preparar el entorno de dos colaboradores	43
9.2.2. Push sin conflicto: flujo básico	43
9.2.3. Push rechazado: dos colaboradores publican a la vez	44
9.2.4. Resolver el rechazo con pull y merge	45
9.2.5. Conflicto entre colaboradores remotos	46
9.2.6. Push forzado seguro con --force-with-lease	47
9.2.7. Limpiar la rama y el entorno	48
9.3. Resultado esperado	48
10. Lab: Configurar organización, grupo y proyecto en GitLab	49
10.1. Objetivos	49
10.2. Prerrequisitos	49
10.3. Ejercicios	49
10.3.1. Verificar el punto de partida	49
10.3.2. Crear un grupo en GitLab	49
10.3.3. Crear un proyecto dentro del grupo	50
10.3.4. Vincular el repositorio local con el nuevo proyecto	50

10.3.5. Configurar la protección de ramas	51
10.3.6. Añadir un miembro al grupo	51
10.3.7. Configurar ajustes adicionales del proyecto	52
10.4. Resultado esperado	52
11. Lab: Gestión de Issues y Milestones (sprint simulado)	53
11.1. Objetivos	53
11.2. Prerrequisitos	53
11.3. Ejercicios	53
11.3.1. Crear labels de grupo	53
11.3.2. Crear un milestone (sprint simulado)	54
11.3.3. Crear issues para el sprint	54
11.3.4. Configurar el tablero de issues	55
11.3.5. Simular el flujo de trabajo del sprint	56
11.4. Resultado esperado	58
12. Lab: Flujo completo de Merge Request con code review en pareja	59
12.1. Objetivos	59
12.2. Prerrequisitos	59
12.3. Ejercicios	59
12.3.1. Verificar el punto de partida	59
12.3.2. Desarrollador A: crear rama y trabajar en el Issue 2	59
12.3.3. Desarrollador A: abrir la Merge Request	60
12.3.4. Revisor: realizar el code review	61
12.3.5. Desarrollador A: responder a la revisión	62
12.3.6. Revisor: aprobar la Merge Request	63
12.3.7. Hacer merge de la MR	63
12.3.8. Desarrollador B: completar el Issue 4 (flujo completo)	64
12.3.9. Cerrar el milestone y verificar el resultado final	65
12.4. Resultado esperado	65
13. Lab: GitLab Duo — extensión VS Code y asistencia IA integrada	67
13.1. Objetivos	67
13.2. Prerrequisitos	67
13.3. Ejercicios	67
13.3.1. Activar la versión de prueba de GitLab Duo	67
13.3.2. Crear un token de acceso personal con scope api	68
13.3.3. Instalar la extensión GitLab for VS Code	68
13.3.4. Crear un script base para los ejercicios con IA	69
13.3.5. GitLab Duo Chat en VS Code	71
13.3.6. Crear un script con ayuda de GitLab Duo y publicarlo	72
13.3.7. Explorar GitLab Duo desde la UI web de GitLab	74
13.3.8. Crear una Merge Request y gestionarla con GitLab Duo	75
13.3.9. GitLab Duo Agent Platform	76

13.3.10. Resolver un conflicto de merge con ayuda de GitLab Duo	79
13.4. Resultado esperado	81

Capítulo 1. Lab: Instalación y configuración de Git

1.1. Objetivos

- Verificar si Git está instalado en el sistema.
- Instalar Git usando el gestor de paquetes de Fedora.
- Configurar la identidad global de Git para commits.
- Definir un editor por defecto para operaciones interactivas.
- Configurar opciones iniciales recomendadas para trabajo diario.
- Validar la configuración efectiva y su origen.
- Crear el repositorio base del proyecto que se utilizará en los siguientes laboratorios.

1.2. Prerrequisitos

- Tener acceso a una terminal con permisos de `sudo`.
- Tener conexión a Internet para instalar paquetes.

1.3. Ejercicios

1.3.1. Comprobación inicial del entorno

Ejecuta los siguientes comandos para comprobar si Git está instalado y qué versión tienes disponible.

```
git --version  
command -v git
```

Si `git --version` no devuelve una versión válida, continúa con el siguiente ejercicio.

1.3.2. Instalación de Git en Fedora

Actualiza metadatos del sistema e instala Git.

```
sudo dnf makecache  
sudo dnf install -y git
```

Vuelve a comprobar la versión instalada.

```
git --version
```

1.3.3. Configuración inicial global

Configura identidad y editor global para el usuario actual. Sustituye los valores por los tuyos.

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@empresa.com"
git config --global core.editor "nano"
```

Define también configuraciones recomendadas de inicio.

```
git config --global init.defaultBranch main
git config --global color.ui auto
git config --global pull.ff only
```

1.3.4. Alias de uso frecuente

Añade alias que se utilizarán durante los siguientes laboratorios.

```
git config --global alias.st status
git config --global alias.logall "log --oneline --graph --decorate --all"
git config --global alias.sw switch
```

1.3.5. Verificación de configuración y origen

Inspecciona la configuración activa y desde qué fichero se carga cada valor.

```
git config --global --list
git config --list
git config --show-origin --list
```

Revisa específicamente los valores clave.

```
git config --global user.name
git config --global user.email
git config --global core.editor
git config --global init.defaultBranch
```

1.3.6. Preparación de un .gitignore base de práctica

Crea el repositorio del proyecto formativo e inicializa una base de trabajo reutilizable para los labs posteriores.

```
mkdir -p ~/workspace/proyecto-control-tareas
cd ~/workspace/proyecto-control-tareas
```



```
git init
```

Crea una estructura mínima del proyecto y un **.gitignore** inicial con patrones comunes.

```
mkdir -p docs src
cat > README.md << 'EOF'
# Proyecto Control de Tareas

Proyecto base para practicar flujo local de Git durante el curso.
EOF

cat > src/tareas.txt << 'EOF'
Tarea inicial: preparar entorno Git
EOF

cat > .gitignore << 'EOF'
*.log
*.tmp
.env
build/
EOF
```

Registra la base inicial del proyecto para usarla como punto de partida en el siguiente laboratorio.

```
git add .
git commit -m "config-lab: inicializar estructura base del proyecto de prácticas"
```

Comprueba el estado e historial del repositorio.

```
git status
git logall
```

1.4. Resultado esperado

Al finalizar el laboratorio, el alumno debe tener:

- Git instalado y operativo.
- Identidad global y editor correctamente configurados.
- Opciones iniciales recomendadas aplicadas (`init.defaultBranch`, `color.ui`, `pull.ff`).
- Alias de uso frecuente disponibles.
- Un proyecto local inicial (`~/workspace/proyecto-control-tareas`) con un primer commit listo para continuar con los siguientes labs.

Capítulo 2. Lab: Ciclo de vida de los ficheros y sistema interno de Git

2.1. Objetivos

- Comprender los estados de un fichero en Git: `untracked`, `modified`, `staged` y `committed`.
- Ver de forma práctica cómo el `index` guarda los cambios preparados.
- Inspeccionar los objetos internos que crea Git: `commit`, `tree` y `blob`.
- Relacionar lo que se ve en `git status` con lo que realmente hay dentro de `.git/`.
- Demostrar que Git registra datos en una estructura concreta y auditable.

2.2. Prerrequisitos

- Tener Git instalado y configurado (identidad global y editor).
- Haber completado el lab anterior y disponer del repositorio `~/workspace/proyecto-control-tareas`.
- Tener acceso a terminal y permisos de escritura en el directorio personal.

2.3. Ejercicios

2.3.1. Retomar el proyecto del laboratorio anterior

Accede al mismo repositorio creado en el lab anterior para mantener una línea de proyecto continua.

```
cd ~/workspace/proyecto-control-tareas
pwd
git logall
```

Comprueba el estado inicial del repositorio.

```
git status
```

2.3.2. Crear un fichero y observar el estado `untracked`

Crea un fichero y observa el cambio de estado.

```
echo "Tarea 2: definir flujo de trabajo" >> src/tareas.txt
echo "Pendiente: revisar prioridades del sprint" > docs/bitacora.txt
git status
```

Ahora `docs/bitacora.txt` estará en estado `untracked` y `src/tareas.txt` en estado `modified`.

2.3.3. Ver dónde está el index y qué contenido tiene

Muestra dónde se guarda el index e inspecciona los elementos actualmente preparados.

```
# Ver la estructura del directorio .git
tree .git

# Ver los archivos rastreables
git ls-files
```

2.3.4. Añadir al staging area y verificar el index

Añade el fichero y comprueba cómo ahora aparece en el index.

```
git add .
git status
git ls-files --stage
```

Ahora Git ha añadido los ficheros sin seguimiento previo al index, y ambos aparecen como preparados para commit.

```
# El fichero index muestra nuevas entradas
file .git/index
```

Hagamos una inspección del historial.

```
git logall
```

Veremos algo similar a esto:

```
* <commit hash> (HEAD -> main) <comentario del commit>
```

Copiamos el hash del commit inicial y lo guardamos en una variable

```
COMMIT_HASH=<hash_del_commit_inicial>
git cat-file -t $COMMIT_HASH
git cat-file -p $COMMIT_HASH
```

2.3.5. Crear un nuevo commit e inspeccionar commit y tree

Confirma los cambios e inspecciona los objetos generados.

```
git commit -m "ciclo-lab: registrar bitácora y actualizar tareas del proyecto"
COMMIT_HASH_2=$(git rev-parse HEAD)
```

```
git cat-file -t "$COMMIT_HASH_2"  
git cat-file -p "$COMMIT_HASH_2"
```

Vamos a comparar el nuevo commit con el anterior para ver la evolución.

```
git cat-file -p "$COMMIT_HASH"
```

Del contenido del commit, identifica el campo **tree** e inspecciona ese objeto. Debemos buscar una línea similar a esta:

```
tree 4b825dc642cb6eb9a060e54bf8d69288e819
```

```
git cat-file -t <identificador_del_tree>  
git cat-file -p <identificador_del_tree>
```

2.3.6. Modificar el fichero para ver la transición modified → staged

Añade una nueva línea y observa la diferencia entre working tree e index.

```
echo "Tarea 3: preparar demo para cliente" >> src/tareas.txt  
git status  
git diff
```

Ahora añade los cambios al staging area y compara.

```
git add src/tareas.txt  
git status  
git diff --staged
```

Añadimos una nueva línea y vemos que el fichero está modificado y también preparado para commit, pero con contenido diferente.

```
echo "Tarea 4: cerrar incidencias abiertas" >> src/tareas.txt  
git status  
  
# Muestra diferencia entre index y working tree  
git diff  
  
# Muestra diferencia entre index y último commit  
git diff --staged
```

Para finalizar esta parte del laboratorio, confirmamos el nuevo commit y revisamos el historial.

```
# Con -a se añaden automáticamente los cambios de los archivos rastreados.  
# sin necesidad de hacer git add previamente.  
git commit -am "ciclo-lab: ampliar backlog técnico del proyecto"  
git logall
```

2.3.7. Navegar por ambos estados

Confirma y revisa la cadena de commits y sus trees.

```
# Muestra el objeto commit actual  
git cat-file -p HEAD  
  
# Muestra el commit anterior  
git cat-file -p HEAD~1
```

2.4. Resultado esperado

Al terminar la práctica, el alumno debe ser capaz de explicar y demostrar que:

- El **index** está en **.git/index** y actúa como staging area real.
- **git add** no es magia: actualiza el index para apuntar a objetos **blob** concretos.
- Un commit referencia un **tree**, y ese **tree** referencia los **blob** de los ficheros.
- El ciclo **modified** → **staged** → **committed** corresponde a cambios reales en objetos internos de Git.

Capítulo 3. Lab: Comandos básicos de trabajo local con Git

3.1. Objetivos

- Retomar el repositorio de prácticas y verificar su estado actual.
- Crear cambios controlados para practicar inspección de historial y diferencias.
- Usar `git log` con formatos y filtros de uso habitual.
- Comparar cambios con `git diff` en distintas fases del trabajo local.
- Inspeccionar commits concretos con `git show`.
- Navegar por el historial usando referencias relativas (`HEAD~N`, `HEAD^`).
- Atribuir cambios línea a línea con `git blame`.
- Crear un clon local del repositorio para una revisión separada.

3.2. Prerrequisitos

- Tener Git instalado y configurado.
- Haber completado los dos laboratorios anteriores.
- Disponer del repositorio `~/workspace/proyecto-control-tareas` con historial previo.
- Trabajar sobre Fedora Linux 41 o un entorno Linux equivalente con terminal disponible.

3.3. Ejercicios

3.3.1. Retomar el proyecto formativo

Accede al repositorio que vienes utilizando desde el inicio del curso y comprueba su estado general.

```
cd ~/workspace/proyecto-control-tareas
git status
git logall
```

Comprueba también cuántos commits existen ya en el proyecto.

```
# Contar el número de commit en la rama actual.
# --count sustituye la salida habitual (lista de hashes) por un número entero.
git rev-list --count HEAD
```

3.3.2. Preparar cambios para inspección

Simula una actualización funcional sencilla del proyecto. Añade una guía de uso y actualiza la descripción del repositorio.

```
cat > docs/guia-uso.md << 'EOF'
```

```
# Guía rápida de uso
```

- Revisar el fichero src/tareas.txt al inicio del día.
- Registrar incidencias técnicas en docs/bitacora.txt.
- Confirmar cambios pequeños y coherentes.

```
EOF
```

```
cat >> README.md << 'EOF'
```

```
## Uso básico
```

```
Este repositorio se utiliza en clase para practicar trabajo local con Git.
```

```
EOF
```

```
git status
```

Revisa la diferencia sin preparar todavía nada.

```
git diff
```

3.3.3. Comparar working tree, staging area y último commit

Prepara solo la nueva guía y observa cómo cambia la salida de Git.

```
git add docs/guia-uso.md
```

```
git status
```

```
git diff
```

```
git diff --staged
```

Ahora añade también el cambio pendiente en **README.md** y crea un commit claro.

```
git add README.md
```

```
git commit -m "basicos-lab: añadir guía de uso y ampliar descripción del proyecto"
```

3.3.4. Leer historial con distintos niveles de detalle

Explora el historial con varios formatos para aprender a localizar información útil con rapidez.

```
# --stat muestra un resumen de los archivos modificados en cada commit.
```

```
# -n limita la salida a los últimos n commits.
```

```
git log --stat -3
```

```
# Mostrar solo los commits que afectan a un archivo específico.
```

```
git log -- docs/guia-uso.md
```

3.3.5. Inspeccionar un commit concreto con git show

Obtén el hash del commit más reciente y revisa exactamente qué cambió.

```
git show HEAD
```

Observa tres elementos: mensaje, metadatos y diff introducido.

3.3.6. Navegar por el historial con referencias relativas

Git permite referirse a commits anteriores sin copiar su hash, usando **referencias relativas** (relative references). Las dos notaciones fundamentales son:

- **HEAD~N** — retrocede N commits siguiendo la cadena de ancestros directos.
- **HEAD^** — selecciona el primer padre de un commit; para el historial lineal es equivalente a **HEAD~1**.

```
# Ver el commit inmediatamente anterior al actual.  
git show HEAD~1
```

```
# Ver el commit de hace dos pasos.  
git show HEAD~2
```

Compara el estado actual con commits anteriores usando la misma notación.

```
# Diferencia entre el árbol actual y el commit previo.  
git diff HEAD~1
```

```
# Diferencia entre dos commits anteriores entre sí.  
git diff HEAD~2 HEAD~1
```

```
# Ver solo los nombres de archivos que cambiaron en los últimos tres commits.  
git diff --name-only HEAD~3..HEAD
```

Filtra el log usando rangos definidos con referencias relativas.

```
# Mostrar los commits desde hace tres pasos hasta HEAD en formato corto.  
git log HEAD~3..HEAD --oneline
```

3.3.7. Usar git blame para rastrear el origen de una línea

Consulta quién modificó por última vez las líneas más recientes de **README.md**.

```
git blame README.md
```


Haz lo mismo sobre el fichero de tareas para ver cómo cada línea queda asociada a un commit.

```
git blame src/tareas.txt
```

3.3.8. Crear un clon local para auditoría rápida

En muchos entornos hace falta revisar un proyecto desde una copia separada para no alterar el árbol de trabajo principal. Crea un clon local del repositorio actual.

```
cd ~/workspace
git clone ~/workspace/proyecto-control-tareas proyecto-control-tareas-auditoria
cd ~/workspace/proyecto-control-tareas-auditoria
git logall
git status
```

Vuelve al repositorio principal al terminar.

```
cd ~/workspace/proyecto-control-tareas
```

3.4. Resultado esperado

Al finalizar el laboratorio, el alumno debe poder:

- Interpretar con soltura `git status`, `git log`, `git diff`, `git show` y `git blame`.
- Distinguir entre cambios en el `working tree`, en el `staging area` y en el último commit.
- Inspeccionar el historial de un proyecto sin depender solo de la vista por defecto.
- Navegar por el historial usando referencias relativas (`HEAD~1`, `HEAD~N`, `HEAD^`) para comparar estados del proyecto sin necesidad de conocer el hash exacto de cada commit.
- Crear un clon local para análisis o validación sin comprometer el repositorio principal.

Capítulo 4. Lab: Stash para guardar trabajo inacabado

4.1. Objetivos

- Identificar los riesgos de cambiar de rama con trabajo local sin confirmar.
- Guardar cambios temporales sin generar commits prematuros.
- Consultar la pila de `stash` y revisar su contenido antes de recuperarlo.
- Aplicar y extraer trabajo temporal con `apply` y `pop`.
- Incluir ficheros no rastreados en un `stash` cuando la situación lo requiera.
- Usar `--keep-index` para aislar solo parte de los cambios locales.
- Mantener el repositorio limpio mientras se atiende una interrupción de trabajo realista.
- Comprender `git worktree` como alternativa al stash para trabajo inacabado de larga duración.
- Crear y eliminar worktrees vinculados para trabajar en ramas paralelas sin cambiar de directorio.

4.2. Prerrequisitos

- Haber completado el laboratorio de comandos básicos.
- Disponer del repositorio `~/workspace/proyecto-control-tareas` con historial local.
- Tener un árbol de trabajo limpio al comenzar este laboratorio.

4.3. Ejercicios

4.3.1. Preparar el escenario de interrupción

Retoma el repositorio principal y asegúrate de que partes de un estado limpio.

```
cd ~/workspace/proyecto-control-tareas
git status
```

Si el clon auxiliar del laboratorio anterior sigue existiendo, no afecta a este ejercicio.

4.3.2. Riesgos de cambiar de rama con trabajo inacabado

NOTE

Este ejercicio usa brevemente el concepto de **rama (branch)** para ilustrar el problema real que `stash` resuelve. Los laboratorios dedicados a ramas profundizarán en todos estos mecanismos; aquí basta con seguir los comandos tal como aparecen.

Crea una rama auxiliar de demostración a partir del estado actual del proyecto.

```
git branch demo-stash
git branch
```

Caso 1: los cambios viajan a la otra rama sin avisar

Cuando los cambios locales no entran en conflicto con la rama de destino, Git permite el cambio y arrastra silenciosamente el trabajo al nuevo contexto.

```
# Modificar un fichero sin prepararlo ni confirmarlo.
echo "nota provisional" >> docs/bitacora.txt
git status
```

```
# Cambiar de rama: Git lo permite y los cambios viajan con nosotros.
git switch demo-stash
git status
```

El peligro es evidente: podrías confirmar ese trabajo en la rama equivocada sin darte cuenta. Vuelve a **main** y deshaz el cambio antes de continuar.

```
git switch main
git restore docs/bitacora.txt
git status
```

Caso 2: Git bloquea el cambio de rama para proteger tus datos

Cuando la rama de destino contiene un estado del fichero que sobrescribiría tu trabajo local sin confirmar, Git rechaza el cambio y describe exactamente qué ficheros están en riesgo.

Primero genera un **commit** (confirmación) en **demo-stash** que modifique el mismo fichero que vas a tocar en **main**.

```
git switch demo-stash
echo "entrada desde demo-stash" >> src/tareas.txt
git add src/tareas.txt
git commit -m "stash-lab: entrada en tareas desde demo-stash"
git switch main
```

Ahora modifica el mismo fichero en **main** sin confirmarlo.

```
echo "entrada local sin confirmar" >> src/tareas.txt
git status
```

Intenta cambiar a **demo-stash**.

```
# Git rechaza el cambio: los datos locales serían sobreescritos.
git switch demo-stash
```

La salida indica los ficheros afectados y sugiere confirmar o guardar los cambios antes de continuar. Este es exactamente el escenario que **git stash** resuelve en los ejercicios siguientes.

Deshaz el cambio local y elimina la rama de demostración para dejar el repositorio limpio.

```
git restore src/tareas.txt
# -D es necesario porque demo-stash tiene commits no fusionados en main.
git branch -D demo-stash
git branch
```

4.3.3. Crear trabajo en curso sin terminar

Simula que estás redactando mejoras de documentación y una nota de despliegue, pero una incidencia urgente te obliga a dejarlo a medias.

```
cat >> README.md << 'EOF'

## Flujo operativo diario

- Revisar tareas abiertas.
- Confirmar cambios pequeños.
- Documentar incidencias al finalizar la jornada.
EOF

cat > docs/despliegue.md << 'EOF'
# Despliegue

Pendiente de completar durante la próxima sesión.
EOF

git add README.md docs/despliegue.md
git status
git diff --staged
```

4.3.4. Guardar el trabajo temporal en un stash con mensaje

Aparta los cambios actuales de forma explícita.

```
git stash push -m "WIP: borrador de flujo operativo y despliegue"
git status
git stash list
```

Inspecciona el contenido del stash antes de continuar.

```
git stash show
```

```
# acepta cualquier formato conocido de git diff.  
git stash show -p stash@{0}
```

4.3.5. Resolver la interrupción urgente

Ahora que el árbol de trabajo está limpio, registra una corrección urgente y coherente en el proyecto.

```
echo "Tarea 5: revisar copia de seguridad semanal" >> src/tareas.txt  
git add src/tareas.txt  
git commit -m "stash-lab: añadir tarea de revisión de copias de seguridad"  
git logall
```

4.3.6. Recuperar los cambios guardados

Aplica el stash sin eliminarlo para verificar el resultado.

```
git stash apply stash@{0}
```

Comprueba que el contenido vuelve al árbol de trabajo y que la entrada sigue en la pila.

```
git stash list
```

Una vez validado, limpia el árbol de trabajo y recupera el mismo stash usando **pop**.

```
git restore README.md  
git restore --staged docs/despliegue.md  
rm docs/despliegue.md  
git stash pop stash@{0}
```

4.3.7. Guardar también archivos no rastreados

Genera un fichero temporal no rastreado y guárdalo junto con el resto del trabajo local.

```
echo "Pendiente: validar checklist de cierre" > docs/pendiente-cierre.txt  
git status  
git stash push -u -m "WIP: incluir borradores no rastreados"  
git status  
git stash list  
git stash show -p stash@{0}
```

Recupera ese trabajo para confirmar que el archivo no rastreado también vuelve.

```
git stash pop stash@{0}  
ls docs  
git status
```

4.3.8. Separar cambios preparados y no preparados con keep-index

Este patrón es útil cuando una parte del trabajo ya está lista para commit y otra parte aún no.

```
git stash push --keep-index -m "WIP: apartar cambios no preparados"  
git status
```

Recupera el cambio apartado al terminar la comprobación.

```
git stash pop stash@{0}
```

Como hemos visto en ejercicios anteriores al recuperar un stash los ficheros que estaban preparados vuelven al estado modificado (excepto los que no estaban rastreados y se añadieron al área de preparación, que vuelven a estar preparados). Existe un modo de devolver los ficheros al estado original en el que fueron apartados.

```
git add README.md  
git stash push -m "WIP: apartar cambios no preparados sin modificar"  
git status  
git stash pop --index stash@{0}  
git status
```

Para cerrar el laboratorio, deja el repositorio limpio sin confirmar estos borradores.

```
git restore --staged README.md docs/despliegue.md  
git restore README.md  
rm -f docs/pendiente-cierre.txt docs/despliegue.md  
git status
```

4.3.9. git worktree como alternativa al stash

NOTE

`git worktree` permite tener **varios directorios de trabajo** sobre el mismo repositorio al mismo tiempo, cada uno apuntando a una rama distinta. Es especialmente útil cuando el trabajo inacabado es suficientemente voluminoso o de larga duración como para que una entrada de stash resulte incómoda de mantener: evita tocar la pila de stash y permite ejecutar pruebas o compilar la rama paralela sin cambiar de directorio.

Verificar el estado inicial

```
cd ~/workspace/proyecto-control-tareas
git status
git branch
```

El árbol de trabajo debe estar limpio y situado en **main**.

Iniciar trabajo inacabado en main

Simula que estás en mitad de una tarea y llega una solicitud urgente en una rama distinta.

```
cat >> README.md << 'EOF'

## Pendiente de completar

- Revisar política de retención de logs.
- Actualizar diagrama de arquitectura.
EOF

git status
```

Los cambios están sin preparar: no quieres perderlos, pero tampoco quieres hacer un commit prematuro ni usar stash.

Crear un worktree vinculado para la tarea urgente

Añade un worktree que apunte a una nueva rama **hotfix/ajuste-tareas**, situado fuera del directorio principal del proyecto.

```
git worktree add ../proyecto-control-tareas-hotfix -b hotfix/ajuste-tareas
```

Comprueba que el directorio existe y lista todos los worktrees activos.

```
ls ../proyecto-control-tareas-hotfix
git worktree list
```

Trabajar en el worktree secundario

Cambia al directorio del worktree secundario y realiza la corrección urgente.

```
cd ../proyecto-control-tareas-hotfix
git branch
git status
```

```
echo "Tarea 6: archivar tareas completadas hace más de 30 días" >> src/tareas.txt
git add src/tareas.txt
git commit -m "hotfix: añadir tarea de archivado periódico"
git logall
```

Comprobar el aislamiento entre worktrees

Vuelve al directorio principal y verifica que tu trabajo inacabado sigue intacto, sin necesidad de stash.

```
cd ~/workspace/proyecto-control-tareas
git status
cat README.md
```

NOTE

El trabajo inacabado en **main** permanece exactamente como lo dejaste. El worktree secundario aísla completamente la rama **hotfix/ajuste-tareas** sin interferir con tu directorio activo.

Integrar el hotfix y limpiar el worktree

Una vez terminada la corrección urgente, fusiona la rama en **main** y elimina el worktree secundario.

```
git merge hotfix/ajuste-tareas --no-ff -m "Merge hotfix/ajuste-tareas en main"
git logall
```

Elimina el worktree secundario y borra la rama ya integrada.

```
git worktree remove ../proyecto-control-tareas-hotfix
git branch -d hotfix/ajuste-tareas
git worktree list
```

Cerrar el trabajo inacabado

Ahora que la urgencia está resuelta, descarta el borrador que quedó en **main**.

```
git restore README.md
git status
```

4.4. Resultado esperado

Al finalizar el laboratorio, el alumno debe ser capaz de:

- Guardar trabajo inacabado con mensajes útiles mediante **git stash push**.
- Consultar y auditar entradas con **git stash list** y **git stash show -p**.

- Diferenciar el comportamiento de `apply`, `pop` y `push -u`.
- Usar `--keep-index` para preservar lo ya preparado mientras aparta el resto.
- Mantener el historial limpio evitando commits temporales sin valor real.
- Crear y gestionar worktrees vinculados con `git worktree add` y `git worktree remove`.
- Identificar cuándo `git worktree` es preferible a `git stash`: trabajo de larga duración, necesidad de ejecutar pruebas en paralelo o cuando se quiere evitar una pila de stash compleja.
- Verificar el aislamiento entre worktrees con `git worktree list` y `git status`.
- Limpiar correctamente los worktrees secundarios tras integrar su rama.

Capítulo 5. Lab: Comandos avanzados de gestión del historial

5.1. Objetivos

- Corregir el último commit local con `git commit --amend`.
- Corregir un commit reciente que incluyó un fichero que no debería entrar en el historial.
- Eliminar artefactos no rastreados con `git clean` usando previsualización.
- Deshacer un commit de forma segura mediante `git revert`.
- Retroceder `HEAD` con control sobre el index y el working tree usando `git reset`.
- Recuperar commits aparentemente perdidos con `git reflog`.
- Reorganizar commits locales con `git rebase -i`.
- Trasladar un commit concreto entre ramas con `git cherry-pick`.
- Distinguir qué operaciones son aptas para historial compartido y cuáles no.

5.2. Prerrequisitos

- Haber completado los laboratorios anteriores del bloque 2.
- Disponer del repositorio `~/workspace/proyecto-control-tareas` en estado limpio.
- Tener configurado un editor funcional para operaciones interactivas de Git.

5.3. Ejercicios

5.3.1. Verificar el punto de partida

Retoma el repositorio y confirma que no hay cambios pendientes antes de trabajar sobre el historial.

```
cd ~/workspace/proyecto-control-tareas
git status
git logall
```

5.3.2. Corregir el último commit con amend

Crea un commit con un mensaje poco preciso y un fichero que debería haber ido incluido desde el principio.

```
cat > docs/checklist-operativo.md << 'EOF'
# Checklist operativo

- Verificar estado del repositorio.
- Revisar cambios antes de confirmar.
```

```
EOF
```

```
git add docs/checklist-operativo.md
git commit -m "hist-lab: cambios"
WRONG_COMMIT=$(git rev-parse HEAD)
```

Ahora añade una mejora olvidada y corrige el mensaje del commit anterior.

```
echo "- Registrar incidencias en la bitácora." >> docs/checklist-operativo.md
git add docs/checklist-operativo.md
git commit --amend -m "hist-lab: añadir checklist operativo inicial"
git rev-parse HEAD
echo $WRONG_COMMIT
```

5.3.3. Corregir un commit que incluyó un fichero indebido

Simula ahora un error más frecuente de lo esperado: añadir al historial un fichero demasiado pesado o con información sensible que no debería formar parte del repositorio.

```
mkdir -p respaldos
cat > respaldos/clientes-export.csv << 'EOF'
id,nombre,correo
1,Ana,ana@example.com
2,Luis,luis@example.com
EOF

git add respaldos/clientes-export.csv
git commit -m "hist-lab: añadir export temporal de clientes"
git logall
```

Asume que detectas el problema justo después del commit y que todavía no lo has publicado a ningún remoto. En ese caso, corrígelo retirando el fichero del índice y rehaciendo el último commit.

```
# Borramos el fichero del index pero no del sistema de archivos.
git rm --cached respaldos/clientes-export.csv
echo "respaldos/" >> .gitignore
git add .gitignore
git commit --amend -m "hist-lab: actualizar materiales sin incluir exportes
temporales"
git status
git logall
git show --stat HEAD
```

Comprueba que el fichero sigue existiendo en tu directorio de trabajo, pero ya no forma parte del historial reciente ni volverá a añadirse por error.

```
ls respaldos
```

```
# Comprobamos si el fichero está siendo ignorado correctamente.
git check-ignore -v respaldos/clientes-export.csv
```

5.3.4. Limpiar artefactos no rastreados con seguridad

Genera directorios y ficheros temporales para simular ruido de trabajo local.

```
mkdir -p tmp/exportes
echo "resultado intermedio" > tmp/exportes/resumen.md
echo "borrador local" > notas-operador.txt
git status
```

Previsualiza qué se eliminaría y ejecuta la limpieza solo cuando lo tengas claro.

```
# dry run
git clean -n
git clean -nd
git clean -ndx
# Ejecutar limpieza (force es necesario para evitar borrados accidentales).
git clean -fd
git status
```

5.3.5. Revertir un cambio sin reescribir historial

Introduce un cambio incorrecto de forma deliberada y reviértelo después con un nuevo commit.

```
echo "ATENCION: todas las tareas deben cerrarse sin revisión." >> README.md
git add README.md
git commit -m "hist-lab: añadir instrucción operativa incorrecta"
git logall
git show --stat HEAD
```

Revierte ese commit y comprueba el resultado.

```
# --no-edit omite la edición del mensaje de revert
git revert HEAD --no-edit
git logall
git show --stat HEAD
```

5.3.6. Deshacer commits locales con reset

Practica los tres modos de `git reset` sobre el repositorio de prácticas para ver cómo afectan de forma diferente al index y al working tree.

Asegúrate de partir de un árbol limpio en `main`.

```
git status
```

Crea dos commits de trabajo que servirán de material para los ejercicios.

```
echo "entrada provisional A" >> src/tareas.txt
git add src/tareas.txt
git commit -m "hist-lab: entrada provisional A"

echo "entrada provisional B" >> src/tareas.txt
git add src/tareas.txt
git commit -m "hist-lab: entrada provisional B"

git logall
```

Modo **--soft**: deshacer el commit conservando los cambios preparados

```
git reset --soft HEAD~1
git status
git diff --staged
```

Los cambios del commit deshecho vuelven al staging area. Puedes confirmarlos de nuevo con un mensaje diferente o ajustar su contenido antes.

```
git commit -m "hist-lab: entrada provisional B (reescrita con --soft)"
git logall
```

Modo **--mixed**: deshacer el commit y vaciar el staging area

```
git reset HEAD~1
git status
git diff
```

Los cambios están de vuelta en el working tree como *modified*, fuera del staging area. Revisa, ajusta si es necesario y vuelve a preparar solo lo que necesites.

```
git add src/tareas.txt
git commit -m "hist-lab: entrada provisional B (reescrita con --mixed)"
git logall
```

Modo **--hard**: descartar commits y cambios completamente

NOTE

Este modo elimina los cambios del working tree. Úsalo únicamente cuando estés seguro de que no necesitas recuperar nada de los commits descartados.

Crea un commit prescindible.

```
echo "cambio que vamos a descartar" >> src/tareas.txt
git add src/tareas.txt
git commit -m "hist-lab: cambio descartable"
git logall
```

Descarta ese commit y verifica que el working tree queda limpio.

```
git reset --hard HEAD~1
git status
git logall
```

5.3.7. Recuperar commits perdidos con reflog

El ejercicio anterior de `--hard` descartó un commit. En la práctica, ese tipo de situaciones pueden ocurrir por accidente. `git reflog` registra todos los movimientos de `HEAD` y permite recuperar esos commits antes de que el garbage collector los elimine.

Consulta el reflog para localizar el commit descartado.

```
git reflog
```

Identifica la entrada correspondiente al commit que eliminaste con `--hard`. Tendrá un aspecto similar a:

```
c4d3a01 HEAD@{N}: commit: hist-lab: cambio descartable
```

Anota el selector `HEAD@{N}` o el hash de ese commit. Compúnalo brevemente para confirmar que es el correcto.

```
git show HEAD@{N}
```

Recupera ese commit crando una rama de rescate apuntando a él, sin mover `main`.

```
git switch -c recuperacion-reflog HEAD@{N}
git logall
```

Verifica que el contenido del commit descartado está de vuelta en esa rama.

```
git show HEAD
```

A continuación vuelve a `main` y elimina la rama de rescate, ya que su propósito era solo de

demostración.

```
git switch main
git branch -D recuperacion-reflog
git logall
```

5.3.8. Reorganizar commits locales con rebase interactivo

Crea una rama de trabajo aislada para practicar reescritura de historial local.

```
git switch -c practica-rebase
echo "Paso 1: revisar cambios locales" > docs/flujo-revision.md
git add docs/flujo-revision.md
git commit -m "hist-lab: crear borrador de flujo de revision"

echo "Paso 2: comprobar diff y estado" >> docs/flujo-revision.md
git add docs/flujo-revision.md
git commit -m "hist-lab: ampliar flujo de revision"

echo "Paso 3: validar antes del commit" >> docs/flujo-revision.md
git add docs/flujo-revision.md
git commit -m "hist-lab: cerrar flujo de revision"

git logall
```

Lanza un rebase interactivo sobre los tres últimos commits y combina los dos últimos en uno solo.

```
git rebase -i HEAD~3
```

En el editor interactivo:

- Observa el orden de los commits listados, que va del **más antiguo al más reciente**.
- Deja el primer commit como **pick**.
- Cambia la segunda y/o tercera línea para usar **squash** en uno de los commits posteriores.
- Guarda y cierra el editor.
- Revisa y, si se solicita, ajusta el mensaje final del commit combinado.

Comprueba el nuevo historial lineal.

```
git logall
```

Si aparece un conflicto, resuélvelo y continúa.

```
git status
```

```
git rebase --continue
```

5.3.9. Trasladar una corrección puntual con cherry-pick

Vuelve a la rama principal, crea una rama auxiliar con una mejora aislada y trae solo ese commit a `main`.

```
git switch main
git switch -c fix-plantilla
cat > docs/plantilla-incidencias.md << 'EOF'
# Plantilla de incidencias

- Fecha:
- Sistema afectado:
- Descripción:
- Acción correctiva:
EOF

git add docs/plantilla-incidencias.md
git commit -m "hist-lab: añadir plantilla de incidencias"
COMMIT_FIX=$(git rev-parse HEAD)
echo "$COMMIT_FIX"

git switch main
git cherry-pick "$COMMIT_FIX"
git logall
```

Deja preparado el repositorio para bloques posteriores eliminando las ramas auxiliares locales si ya no se necesitan.

```
git branch
git status
```

Si quieres borrar esas ramas al terminar, revisa antes que no necesites conservar su historial local y elimínalas de forma consciente en una sesión posterior.

5.4. Resultado esperado

Al terminar el laboratorio, el alumno debe poder explicar y demostrar que:

- `git commit --amend` corrige el último commit local reescribiendo su identidad.
- Un fichero añadido por error al último commit puede retirarse del índice con `git rm --cached` y corregirse con `git commit --amend` si todavía no se ha publicado.
- `git clean -n` permite validar una limpieza antes de ejecutar `git clean -fd`.
- `git revert` deshace cambios de forma segura creando un nuevo commit compensatorio.
- `git reset --soft` retrocede `HEAD` dejando los cambios en el staging area.

- `git reset --mixed` (por defecto) retrocede `HEAD` y vacía el staging area, conservando los cambios en el working tree.
- `git reset --hard` descarta completamente los commits y los cambios del working tree.
- `git reflog` registra todos los movimientos de `HEAD` localmente y permite recuperar commits aparentemente perdidos.
- `git rebase -i` sirve para reorganizar historial local no compartido.
- `git cherry-pick` traslada un cambio concreto sin fusionar una rama completa.

Capítulo 6. Lab: Flujo feature branch con conflicto y resolución

6.1. Objetivos

- Crear y gestionar ramas de funcionalidad siguiendo la convención `feature/`.
- Observar la diferencia entre un merge fast-forward y un merge commit.
- Provocar un conflicto de contenido de forma controlada y resolverlo manualmente.
- Gestionar un conflicto de tipo «eliminado versus modificado» (`deleted by them`).
- Abandonar un merge conflictivo con `git merge --abort` y recuperar el estado previo.
- Resolver un conflicto sin edición manual usando `git checkout --ours / --theirs`.
- Obtener un historial con ramas claramente identificables usando `git logall`.
- Practicar la limpieza de ramas una vez integradas.

6.2. Prerrequisitos

- Disponer del repositorio `~/workspace/proyecto-control-tareas` en estado limpio en la rama `main`.
- Tener configurado un editor funcional.

6.3. Ejercicios

6.3.1. Verificar el punto de partida

Retoma el repositorio y confirma que el árbol de trabajo está limpio antes de trabajar con ramas.

```
cd ~/workspace/proyecto-control-tareas
git status
git branch
git logall
```

6.3.2. Merge fast-forward: rama sin conflicto

Crea una rama de funcionalidad para añadir un nuevo documento al proyecto.

```
git switch -c feature/panel-de-seguimiento
```

Desarrolla la funcionalidad en dos commits pequeños.

```
cat > docs/panel-seguimiento.md << 'EOF'
# Panel de seguimiento

## Tareas en curso
```

```
| Tarea | Responsable | Estado |  
|-----|-----|-----|  
| Revisar backlog | Equipo | Pendiente |  
EOF
```

```
git add docs/panel-seguimiento.md  
git commit -m "ramas-lab: crear estructura del panel de seguimiento"
```

```
echo "" >> docs/panel-seguimiento.md  
echo "## Notas de última sesión" >> docs/panel-seguimiento.md  
echo "- Primera revisión del panel completada." >> docs/panel-seguimiento.md  
git add docs/panel-seguimiento.md  
git commit -m "ramas-lab: añadir sección de notas al panel"  
  
git logall
```

Fusiona en **main**. Como **main** no ha avanzado, el merge será fast-forward.

```
git switch main  
git merge feature/panel-de-seguimiento  
git logall
```

Observa que no hay merge commit: el puntero de **main** simplemente se ha adelantado. Elimina la rama porque ya está integrada.

```
git branch -d feature/panel-de-seguimiento  
git logall
```

6.3.3. Preparar un conflicto entre ramas

Este ejercicio simula una situación habitual: dos líneas de trabajo editan el mismo fichero de forma incompatible.

Crea una rama de funcionalidad que modifica el **README.md**.

```
git switch -c feature/seccion-contacto  
cat >> README.md << 'EOF'  
  
## Contacto  
  
Para incidencias urgentes, usar el canal #operaciones en el chat interno.  
EOF  
  
git add README.md  
git commit -m "ramas-lab: añadir sección de contacto al README"
```

Sin cerrar esa rama, vuelve a `main` y realiza también un cambio sobre `README.md`.

```
git switch main
cat >> README.md << 'EOF'

## Soporte

Para soporte técnico, abrir una incidencia en el sistema de tickets.
EOF

git add README.md
git commit -m "ramas-lab: añadir sección de soporte al README"
git logall
```

Ahora `main` y `feature/seccion-contacto` han divergido: ambas modificaron el final de `README.md`.

6.3.4. Provocar y resolver el conflicto

Intenta fusionar la rama de funcionalidad en `main`.

```
git merge feature/seccion-contacto
```

Git detecta el conflicto y se detiene. Comprueba qué ficheros están en conflicto.

```
git status
```

Abre `README.md` y localiza los marcadores de conflicto. Tendrá un aspecto similar a:

```
<<<<<< HEAD
## Soporte

Para soporte técnico, abrir una incidencia en el sistema de tickets.
=====
## Contacto

Para incidencias urgentes, usar el canal #operaciones en el chat interno.
>>>>>> feature/seccion-contacto
```

Edita el fichero para conservar ambas secciones en el orden adecuado, sin marcadores:

```
# Editar README.md con el editor configurado, dejando ambas secciones.
code README.md
```

Marca el fichero como resuelto y completa el merge.

```
git add README.md
git commit -m "ramas-lab: merge feature/seccion-contacto con resolución de conflicto"
git logall
```

6.3.5. Merge --no-ff explícito

Crea una última rama que añada un fichero de métricas y fusiónala en `main` forzando un merge commit aunque sea posible el fast-forward.

```
git switch -c feature/metricas-operativas
cat > docs/metricas.md << 'EOF'
# Métricas operativas

- Tiempo medio de resolución de incidencias: pendiente de medir.
- Número de tareas completadas esta semana: 0.
EOF

git add docs/metricas.md
git commit -m "ramas-lab: añadir fichero de métricas operativas"
git switch main
git merge --no-ff feature/metricas-operativas -m "ramas-lab: merge feature/metricas-operativas"
git logall
```

Compara el historial resultante con el del merge fast-forward del primer ejercicio: ahora se ve claramente la bifurcación aunque `main` no había avanzado.

6.3.6. Conflicto por fichero eliminado en una rama y modificado en otra

Un caso frecuente en equipos es que una rama elimine un fichero que otra rama ha modificado. Git no puede decidir automáticamente si el fichero debe conservarse o desaparecer, por lo que detiene el merge y muestra el estado `deleted by them` (o `deleted by us`).

Crea una rama que retira el documento del panel de seguimiento.

```
git switch -c feature/retirar-panel
git rm docs/panel-seguimiento.md
git commit -m "ramas-lab: retirar panel de seguimiento obsoleto"
```

Sin fusionar esa rama, vuelve a `main` y añade contenido al mismo fichero.

```
git switch main
cat >> docs/panel-seguimiento.md << 'EOF'

## Acciones de seguimiento

- Priorizar tareas del sprint actual.
```

```
EOF
```

```
git add docs/panel-seguimiento.md
git commit -m "ramas-lab: añadir acciones de seguimiento al panel"
git logall
```

Intenta fusionar la rama que elimina el fichero.

```
git merge feature/retirar-panel
git status
```

Git indica **deleted by them** en **docs/panel-seguimiento.md**. Tienes dos opciones:

- **Conservar** el fichero con los cambios de **main**: `git add docs/panel-seguimiento.md`
- **Aceptar la eliminación**: `git rm docs/panel-seguimiento.md`

En este caso se decide conservar el fichero.

```
# Conservar el fichero con nuestros cambios
git add docs/panel-seguimiento.md
git commit -m "ramas-lab: conservar panel con acciones tras merge"
git logall
```

6.3.7. Abandonar un merge conflictivo

Cuando se inicia un merge y los conflictos son demasiado complejos para resolverse en ese momento, `git merge --abort` devuelve el repositorio exactamente al estado previo al intento de fusión.

Crea una rama que modifica el fichero de métricas.

```
git switch -c feature/nuevo-indicador
cat >> docs/metricas.md << 'EOF'
- Número de incidencias abiertas esta semana: 0.
EOF

git add docs/metricas.md
git commit -m "ramas-lab: añadir indicador de incidencias a métricas"
```

Vuelve a **main** y realiza un cambio incompatible en el mismo fichero.

```
git switch main
cat >> docs/metricas.md << 'EOF'
- Satisfacción del equipo (escala 1-5): pendiente de encuesta.
EOF

git add docs/metricas.md
```

```
git commit -m "ramas-lab: añadir indicador de satisfacción del equipo"
git logall
```

Inicia el merge y, ante el conflicto, abórtalo.

```
git merge feature/nuevo-indicador # se detiene en conflicto
git status                        # muestra ficheros en conflicto
git merge --abort                 # abandona y restaura el estado previo
git status                        # árbol de trabajo limpio
git logall                       # main sigue en el mismo commit
```

Comprueba que **main** sigue en el mismo **commit** (confirmación) que antes del intento de merge. El árbol de trabajo queda limpio y la rama **feature/nuevo-indicador** permanece intacta.

6.3.8. Resolver un conflicto aceptando íntegramente una versión

Cuando se sabe de antemano qué versión debe prevalecer, **git checkout --ours** o **git checkout --theirs** evitan la edición manual marcador a marcador.

Retoma el merge que se abortó en el ejercicio anterior.

```
git merge feature/nuevo-indicador
git status
```

En lugar de editar manualmente, acepta íntegramente la versión de la rama entrante.

```
git checkout --theirs docs/metricas.md # toma la versión de feature/nuevo-indicador
git add docs/metricas.md
git commit -m "ramas-lab: merge feature/nuevo-indicador con versión de la rama"
git logall
```

La opción complementaria **--ours** habría conservado la versión de **main**, descartando los cambios de la rama entrante.

NOTE

Para aplicar esta estrategia a **todos** los ficheros en conflicto de un merge de forma automática, usa la opción de estrategia directamente en el comando de fusión: **git merge -X theirs** o **git merge -X ours**. Úsalas con precaución: descartan íntegramente el lado opuesto sin aviso.

6.3.9. Merge con squash: condensar commits de una rama en uno solo

Cuando una rama de funcionalidad acumula commits de trabajo en curso — mensajes **wip:**, correcciones de typos, pasos experimentales — el **squash merge** (fusión con aplastamiento) permite integrar todos esos cambios en *un único commit* sobre **main**, manteniendo el historial limpio.

Crea una rama con varios commits pequeños que simulan trabajo incremental.

```
git switch -c feature/plantilla-informe

# Primer commit: estructura básica
cat > docs/plantilla-informe.md << 'EOF'
# Plantilla de informe
EOF
git add docs/plantilla-informe.md
git commit -m "wip: crear fichero plantilla-informe" ①

# Segundo commit: sección de resumen
cat >> docs/plantilla-informe.md << 'EOF'

## Resumen ejecutivo

Pendiente de redactar.
EOF
git add docs/plantilla-informe.md
git commit -m "wip: draft sección resumen ejecutivo"

# Tercer commit: tabla de incidencias
cat >> docs/plantilla-informe.md << 'EOF'

## Detalle de incidencias

| ID | Descripción | Estado |
|----|-----|-----|
EOF
git add docs/plantilla-informe.md
git commit -m "wip: draft tabla de incidencias"

git logall
```

① Los mensajes **wip**: (work in progress) son habituales durante el desarrollo, pero no aptos para el historial permanente de **main**.

Vuelve a **main** y aplica el squash merge. Git traerá todos los cambios al índice pero *no creará ningún commit automáticamente*.

```
git switch main
git merge --squash feature/plantilla-informe
git status      # los ficheros aparecen como "Changes to be committed"
```

Crea el commit definitivo con un mensaje limpio y representativo.

```
git commit -m "feat: añadir plantilla de informe con resumen e incidencias"
git logall
```


Compara el resultado con el de un merge convencional: en el historial aparece un único commit en **main**, sin rastro de los tres commits intermedios de la rama y sin commit de merge de dos padres.

IMPORTANT

`git merge --squash` **no registra la fusión** como un merge de dos padres. Git no «recuerda» que `feature/plantilla-informe` está integrada: `git branch -d` advierte de que la rama no ha sido fusionada. Usa `git branch -D` para eliminarla.

```
git branch -d feature/plantilla-informe # aviso: "not fully merged"
git branch -D feature/plantilla-informe # forzar eliminación
git logall
```

6.3.10. Limpiar ramas integradas

Elimina las ramas de funcionalidad que ya están integradas en **main**.

```
git branch
git branch -d feature/seccion-contacto feature/metricas-operativas \
    feature/retirar-panel feature/nuevo-indicador
git branch
git logall
```

6.4. Resultado esperado

Al terminar el laboratorio, el alumno debe poder:

- Crear y eliminar ramas de funcionalidad siguiendo la convención `feature/`.
- Distinguir en el historial un merge fast-forward de un merge commit.
- Provocar un conflicto de contenido intencionadamente y resolverlo aplicando el flujo correcto: editar → `git add` → `git commit`.
- Usar `git merge --no-ff` para preservar la trazabilidad de trabajo realizado en rama.
- Identificar y resolver un conflicto de tipo «eliminado versus modificado» eligiendo entre conservar el fichero (`git add`) o aceptar su eliminación (`git rm`).
- Abandonar una fusión en curso con `git merge --abort` y verificar que el repositorio queda en el estado previo.
- Resolver un conflicto sin edición manual usando `git checkout --ours` / `git checkout --theirs` y conocer la variante `git merge -X`.
- Usar `git merge --squash` para condensar el historial de una rama en un único commit y comprender sus diferencias respecto a un merge convencional.
- Mantener el repositorio limpio eliminando ramas integradas.

Capítulo 7. Lab: Etiquetado de release con versionado semántico

7.1. Objetivos

- Crear tags ligeras y tags anotadas sobre el repositorio de prácticas.
- Identificar las diferencias entre ambos tipos de tags.
- Aplicar el esquema de versionado semántico (SemVer) para nombrar una release.
- Gestionar el ciclo de vida de las tags: listar, inspeccionar y eliminar.
- Comprender cómo se publicarían las tags a un remoto (demostración conceptual).

7.2. Ejercicios

7.2.1. Verificar el punto de partida

Confirma que el repositorio está en condiciones de etiquetar.

```
cd ~/workspace/proyecto-control-tareas
git status
git logall
git tag
```

El repositorio no debe tener tags todavía y el árbol debe estar limpio.

7.2.2. Crear una tag ligera

Crea una tag ligera para marcar el estado actual como candidato de release.

```
git tag v1.0.0-rc1
git tag
```

Inspecciona a qué apunta la tag ligera.

```
# Una tag ligera apunta directamente al commit
git rev-parse v1.0.0-rc1
git rev-parse HEAD

# Comprueba que son el mismo hash
git cat-file -t v1.0.0-rc1
```

7.2.3. Crear una tag anotada para la release oficial

Crea la tag anotada que marcará la primera versión estable del proyecto de prácticas.

```
git tag -a v1.0.0 -m "Release v1.0.0: versión inicial estable del proyecto de prácticas"
git tag
```

Inspecciona el objeto tag anotado y compara con la tag ligera.

```
# La tag anotada tiene su propio objeto en la base de datos
git cat-file -t v1.0.0
git cat-file -p v1.0.0
```

```
# git show muestra el objeto tag y el diff del commit al que apunta
git show v1.0.0
```

Observa los campos: **tagger**, fecha y mensaje están presentes en la tag anotada, no en la ligera.

7.2.4. Listar y filtrar tags

Practica la gestión básica del listado de tags.

```
# Listar todas las tags
git tag

# Filtrar por patrón
git tag -l "v1.*"
git tag -l "*rc*"
```

Muestra el historial con las tags visibles.

```
git logall
```

Las tags deben aparecer junto a sus respectivos commits en el formato (**tag: v1.0.0**).

7.2.5. Etiquetar un commit anterior

Crea una tag anotada sobre el primer commit del repositorio para simular un etiquetado retroactivo.

```
# Obtener el hash del primer commit del proyecto
#rev-list muestra los SHAs que cumplen un criterio
PRIMER_COMMIT=$(git rev-list --max-parents=0 HEAD)

echo "$PRIMER_COMMIT"
git show --oneline --no-patch "$PRIMER_COMMIT"
```

```
git tag -a v0.1.0 "$PRIMER_COMMIT" -m "Release v0.1.0: estructura inicial del
proyecto"
git tag
git logall
```

7.2.6. Limpiar la tag de release candidate

La tag ligera `v1.0.0-rc1` ya no es necesaria ahora que existe la release oficial.

```
git tag -d v1.0.0-rc1
git tag
```

7.2.7. Publicación de tags al remoto (conceptual)

Este repositorio aún no tiene un remoto configurado (eso se hará en el Bloque 4). Sin embargo, es importante conocer los comandos de publicación para aplicarlos en cuanto el remoto esté disponible.

```
# Publican la tag anotada de release al remoto (ejecutar cuando haya remoto):
# git push origin v1.0.0

# Publican solo tags anotadas alcanzables (recomendado en pipelines):
# git push --follow-tags

# Publicar todas las tags (incluye ligeras, usar con criterio):
# git push origin --tags
```

NOTE

En el Bloque 4 se pondrá en práctica el push de tags al configurar el primer remoto en GitLab.

7.3. Resultado esperado

Al terminar el laboratorio, el alumno debe poder:

- Crear tags ligeras y anotadas sobre cualquier commit del repositorio.
- Explicar por qué las tags anotadas son preferibles para releases oficiales.
- Nombrar correctamente una versión usando el esquema SemVer (**MAJOR.MINOR.PATCH**).
- Listar, filtrar, inspeccionar y eliminar tags localmente.
- Identificar el comando correcto para publicar tags al remoto cuando el repositorio esté conectado.

Capítulo 8. Lab: Configuración SSH y primer push a GitLab

8.1. Objetivos

- Generar un par de claves SSH Ed25519 en Fedora Linux y registrar la clave pública en GitLab.
- Crear un nuevo proyecto en GitLab para albergar el repositorio local.
- Configurar el remoto **origin** en el repositorio de prácticas.
- Realizar el primer **push** y verificar que los commits y tags aparecen en GitLab.
- Crear un Personal Access Token (PAT) y comprobar el acceso HTTPS como alternativa.

8.2. Prerrequisitos

- Disponer del repositorio `~/workspace/proyecto-control-tareas` en estado limpio con las tags creadas en el laboratorio anterior.
- Tener acceso a una instancia de GitLab con permisos para crear proyectos.
- Fedora Linux con **openssh-clients** instalado (incluido por defecto).

8.3. Ejercicios

8.3.1. Verificar el punto de partida

```
cd ~/workspace/proyecto-control-tareas
git status
git logall
git tag
```

El repositorio debe estar limpio, con las tags **v0.1.0** y **v1.0.0** del laboratorio anterior.

8.3.2. Generar el par de claves SSH

Comprueba si ya existe una clave SSH.

```
ls -la ~/.ssh/
```

Si no hay ficheros **id_ed25519** / **id_ed25519.pub**, genera un par nuevo.

```
ssh-keygen -t ed25519 -C "mi-clave-ssh-para-gitlab"
```

Acepta la ruta por defecto (`~/.ssh/id_ed25519`) y establece una passphrase segura cuando se solicite.

Verifica que los ficheros se crearon correctamente y tienen los permisos adecuados.

```
ls -la ~/.ssh/id_ed25519*  
# El fichero privado debe tener permisos 600, el público 644
```

8.3.3. Registrar la clave pública en GitLab

Muestra la clave pública para copiarla.

```
cat ~/.ssh/id_ed25519.pub
```

En GitLab:

1. Pulsar en el **avatar** (esquina superior derecha) → **Edit Profile**.
2. En el menú lateral, seleccionar **Access** → **SSH Keys**.
3. Pulsar **Add new key**.
4. Pegar la clave pública, asignar un título descriptivo (p. ej. «Fedora - portátil de prácticas»).
5. Opcionalmente, establecer una fecha de expiración.
6. Pulsar **Add key**.

8.3.4. Verificar la conexión SSH con GitLab

Arranca el agente SSH y añade la clave.

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

Podemos establecer un alias para facilitar la ejecución de ssh-agent en futuras sesiones, pero no es obligatorio. Editamos el fichero `~/.zshrc` (o `~/.bashrc` si usas Bash) y añadimos la siguiente línea al final:

zshrc

```
alias st-agent='eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519'
```

Comprueba la conexión con el servidor GitLab.

```
ssh -T git@gitlab.com
```

Es posible que aparezca un mensaje de advertencia sobre el host desconocido. Acepta la conexión para añadir la clave del servidor a `~/.ssh/known_hosts`.

Si todo está correcto, GitLab responde con un mensaje de bienvenida que incluye tu nombre de usuario.

8.3.5. Crear un nuevo proyecto en GitLab

En la interfaz web de GitLab:

1. Pulsar **New project** → **Create blank project**.
2. Nombre del proyecto: **proyecto-control-tareas**.
3. Visibilidad: **Private** (privado).
4. **Desmarcar** la casilla «Initialize repository with a README» (el repositorio ya existe en local con historial).
5. Pulsar **Create project**.

GitLab muestra la pantalla con las instrucciones para un repositorio existente. Copiar la URL SSH del proyecto.

8.3.6. Configurar el remoto y hacer el primer push

Añade el remoto **origin** al repositorio local.

```
# Sustituir la URL por la real del proyecto creado en GitLab
git remote add origin git@gitlab.ejemplo.com:tu-usuario/proyecto-control-tareas.git
git remote -v
```

Sube todas las ramas y tags al remoto.

```
git push -u origin main
git push origin --tags
```

Verifica en la interfaz web de GitLab que:

- Los commits aparecen en **Code** → **Repository** → **Commits**.
- Las tags **v0.1.0** y **v1.0.0** aparecen en **Code** → **Repository** → **Tags**.

Confirma que la rama upstream quedó establecida.

```
git branch -vv
```

8.3.7. Crear un Personal Access Token (ejercicio complementario)

Como ejercicio complementario, crea un PAT para comprobar el acceso HTTPS.

En GitLab:

1. **Avatar** → **Edit Profile** → **Access** → **Personal Access Tokens**.
2. **Generate token** → **Legacy token**.
3. Nombre: **practica-pat**.

4. Elegir fecha de expiración (p. ej. 1 semana).
5. Scopes: `read_repository`, `write_repository`.
6. Pulsar **Create personal access token** y copiar el valor.

Verifica que funciona clonando el repositorio por HTTPS en un directorio temporal.

```
cd /tmp
git clone https://gitlab.ejemplo.com/tu-usuario/proyecto-control-tareas.git pat-test
# Usuario: tu-usuario
# Contraseña: <pegar el PAT>
ls pat-test/
rm -rf pat-test
cd ~/workspace/proyecto-control-tareas
```

8.4. Resultado esperado

Al terminar el laboratorio, el alumno debe poder:

- Generar un par de claves SSH Ed25519 y registrar la clave pública en GitLab.
- Verificar la conexión SSH con `ssh -T`.
- Crear un proyecto vacío en GitLab y conectarlo como remoto `origin`.
- Publicar commits y tags en GitLab con `git push`.
- Crear y utilizar un PAT para acceso HTTPS como alternativa a SSH.

Capítulo 9. Lab: Colaboración simulada con fetch, pull y push

9.1. Objetivos

- Simular el trabajo de dos colaboradores sobre el mismo repositorio remoto.
- Comprender la diferencia entre `git fetch` y `git pull` en la práctica.
- Gestionar un push rechazado y resolverlo con el flujo `pull` → `resolve` → `push`.
- Resolver un conflicto originado entre dos colaboradores que modifican el mismo fichero.
- Utilizar `git push --force-with-lease` de forma segura en una rama personal.

9.2. Ejercicios

9.2.1. Preparar el entorno de dos colaboradores

Para simular dos personas trabajando a la vez, clona el repositorio desde GitLab en un segundo directorio.

```
cd ~/workspace
git clone git@gitlab.ejemplo.com:tu-usuario/proyecto-control-tareas.git colaborador-b
ls colaborador-b/
```

A partir de ahora:

- `~/workspace/proyecto-control-tareas` es el directorio del **Colaborador A** (tú mismo).
- `~/workspace/colaborador-b` es el directorio del **Colaborador B** (compañero simulado).

9.2.2. Push sin conflicto: flujo básico

El Colaborador B realiza un cambio y lo publica.

```
cd ~/workspace/colaborador-b

cat > docs/guia-despliegue.md << 'EOF'
# Guía de despliegue

## Entorno de desarrollo

1. Clonar el repositorio.
2. Instalar dependencias.
3. Ejecutar las pruebas.
EOF

git add docs/guia-despliegue.md
git commit -m "remotos-lab: crear guía de despliegue (colaborador B)"
```

```
git push
```

El Colaborador A no tiene ese commit todavía. Comprueba el estado con **fetch** antes de integrar.

```
cd ~/workspace/proyecto-control-tareas
git fetch origin
```

Observa que la rama de seguimiento remoto ha avanzado pero la rama local no.

```
# Comparar la rama local con la remota
git log main..origin/main --oneline

# Ver el diff de los cambios nuevos
git diff main origin/main
```

Integra los cambios en la rama local.

```
git pull origin main
git logall
ls docs/guia-despliegue.md
```

9.2.3. Push rechazado: dos colaboradores publican a la vez

El Colaborador A hace un commit y lo publica.

```
cd ~/workspace/proyecto-control-tareas
echo "- Verificación de despliegue completada." >> docs/guia-despliegue.md
git add docs/guia-despliegue.md
git commit -m "remotos-lab: añadir paso de verificación a la guía (colaborador A)"
git push
```

Sin actualizar, el Colaborador B también modifica (una línea diferente) y sube un cambio al mismo fichero.

Editamos el fichero de colaborador B para que tenga este aspecto: `.docs/guia-despliegue.md`

```
# Guía de despliegue

## Pasos a seguir en entorno Desarrollo

1. Clonar el repositorio.
2. Instalar dependencias.
3. Ejecutar las pruebas.
```

Hacemos commit y push sin actualizar.

```
git add docs/guia-despliegue.md
git commit -m "remotos-lab: modificar título sección desarrollo (colaborador B)"
git push
```

El push es **rechazado** porque **origin/main** ha avanzado con el commit de A. Git muestra un mensaje como:

```
! [rejected]        main -> main (fetch first)
```

Esto es el comportamiento esperado: Git protege el historial del remoto.

9.2.4. Resolver el rechazo con pull y merge

El Colaborador B necesita actualizar su rama local antes de poder publicar.

```
cd ~/workspace/colaborador-b
git pull origin main
```

Se han editado dos líneas diferentes y que no están próximas por lo que no deberían producirse conflictos.

Sin embargo al haber divergencias y tener configurado el pull como fast-forward, Git no puede avanzar la rama local sin crear un merge commit. Veremos un mensaje como:

```
hint: Diverging branches can't be fast-forwarded, you need to either:
hint:
hint:   git merge --no-ff
hint:
hint: or:
hint:
hint:   git rebase
hint:
hint: Disable this message with "git config set advice.diverging false"
fatal: Not possible to fast-forward, aborting.
```

El colaborador B resuelve el merge, confirmando el mensaje por defecto del commit de merge. Y puede hacer push sin problemas.

```
git push origin main
```

Ahora el Colaborador A puede traerse los cambios.

```
cd ~/workspace/proyecto-control-tareas
git pull origin main
```

```
git logall
```

9.2.5. Conflicto entre colaboradores remotos

Ahora ambos colaboradores editan la **misma línea** del mismo fichero para provocar un conflicto.

El Colaborador A modifica la primera línea del README.

```
cd ~/workspace/proyecto-control-tareas
sed -i '1s/.*/# Proyecto de control de tareas – Equipo Alfa/' README.md
git add README.md
git commit -m "remotos-lab: renombrar proyecto a Equipo Alfa (colaborador A)"
git push
```

El Colaborador B, sin actualizar, modifica la misma línea.

```
cd ~/workspace/colaborador-b
sed -i '1s/.*/# Proyecto de control de tareas – Equipo Beta/' README.md
git add README.md
git commit -m "remotos-lab: renombrar proyecto a Equipo Beta (colaborador B)"
git push
```

El push es rechazado. El Colaborador B hace pull.

```
git pull origin main
```

Git se detiene en conflicto. Comprueba el estado.

```
git status
```

Abre el fichero y localiza los marcadores de conflicto.

```
head -10 README.md
```

Tendrá un aspecto similar a:

```
<<<<<<< HEAD
# Proyecto de control de tareas – Equipo Beta
=====
# Proyecto de control de tareas – Equipo Alfa
>>>>>>> abc1234...
```

Edita para dejar un nombre consensuado, eliminando los marcadores de conflicto.

```
# Editar README.md con el editor configurado
code README.md
```

Marca como resuelto y completa el flujo.

```
git add README.md
git commit -m "remotos-lab: resolver conflicto de nombre del proyecto"
git push
```

El Colaborador A se sincroniza.

```
cd ~/workspace/proyecto-control-tareas
git pull origin main
git logall
head -1 README.md
```

9.2.6. Push forzado seguro con `--force-with-lease`

El Colaborador A reescribe el último commit con `--amend` en una rama personal y necesita forzar el push.

```
cd ~/workspace/proyecto-control-tareas
git switch -c feature/doc-arquitectura

cat > docs/arquitectura.md << 'EOF'
# Arquitectura del proyecto

Documento en borrador.
EOF

git add docs/arquitectura.md
git commit -m "remotos-lab: borrador de arquitectura"
git push -u origin feature/doc-arquitectura
```

Corrige el commit con `--amend` (reescritura local del historial).

```
echo "" >> docs/arquitectura.md
echo "## Componentes principales" >> docs/arquitectura.md
echo "" >> docs/arquitectura.md
echo "- Módulo de tareas" >> docs/arquitectura.md
echo "- Módulo de seguimiento" >> docs/arquitectura.md

git add docs/arquitectura.md
git commit --amend -m "remotos-lab: documento de arquitectura con componentes"
```

Un push normal es rechazado porque el historial local difiere del remoto.

```
git push
```

Usa `--force-with-lease` para forzar de forma segura.

```
git push --force-with-lease origin feature/doc-arquitectura
```

9.2.7. Limpiar la rama y el entorno

Fusiona la rama en `main`, limpia local y remoto, y sincroniza el Colaborador B.

```
git switch main
git merge --no-ff feature/doc-arquitectura -m "remotos-lab: merge feature/doc-
arquitectura"
git push

git branch -d feature/doc-arquitectura
git push origin --delete feature/doc-arquitectura
# También sirve: git push -d origin feature/doc-arquitectura

git logall
```

Sincroniza el Colaborador B para dejarlo actualizado.

```
cd ~/workspace/colaborador-b
git pull origin main
git logall
```

9.3. Resultado esperado

Al terminar el laboratorio, el alumno debe poder:

- Usar `git fetch` para descargar cambios sin integrarlos y `git pull` para integrarlos.
- Identificar un push rechazado y resolverlo con el flujo `pull` → `resolver` → `push`.
- Resolver conflictos originados por ediciones simultáneas de dos colaboradores.
- Usar `git push --force-with-lease` de forma segura en una rama personal tras un `--amend` o `rebase`.
- Eliminar ramas remotas una vez integradas con `git push origin --delete`.

Capítulo 10. Lab: Configurar organización, grupo y proyecto en GitLab

10.1. Objetivos

- Crear un grupo en GitLab que represente una organización o equipo de trabajo.
- Configurar la visibilidad del grupo y entender sus implicaciones.
- Crear un proyecto dentro del grupo con la configuración adecuada.
- Configurar la protección de ramas para impedir push directo a `main`.
- Añadir un compañero como miembro del grupo y verificar los permisos heredados.

10.2. Prerrequisitos

- Disponer del repositorio `~/workspace/proyecto-control-tareas` sincronizado con GitLab (Labs anteriores completados).
- Tener acceso a una instancia de GitLab con permisos para crear grupos y proyectos.

10.3. Ejercicios

10.3.1. Verificar el punto de partida

Antes de comenzar, asegúrate de que el repositorio de prácticas está limpio y sincronizado.

```
cd ~/workspace/proyecto-control-tareas
git status
git logall
```

10.3.2. Crear un grupo en GitLab

Accede a la instancia de GitLab en el navegador y crea un nuevo grupo que represente a tu empresa o equipo.

1. Haz clic en **Menu > Groups > Create group** (o el botón **+ > New group**).
2. Selecciona **Create group**.
3. Completa la configuración:

Campo	Valor sugerido
Group name	<code>empresa-formacion</code>
Group URL	<code>empresa-formacion</code> (se genera automáticamente)
Visibility level	Private (privado)"

4. Haz clic en **Create group**.

10.3.3. Crear un proyecto dentro del grupo

Ahora crea un proyecto nuevo dentro del grupo. Este proyecto servirá como repositorio principal del equipo.

1. Dentro del grupo **empresa-formacion**, haz clic en **New project**.
2. Selecciona **Create blank project**.
3. Completa la configuración:

Campo	Valor sugerido
Project name	control-tareas-equipo
Project URL	Verifica que el namespace es empresa-formacion
Visibility level	Private
Initialize repository with a README	Desactivado (vamos a subir el repositorio existente)

4. Haz clic en **Create project**.

GitLab mostrará las instrucciones para subir un repositorio existente. Las usaremos a continuación.

10.3.4. Vincular el repositorio local con el nuevo proyecto

Ahora vincularemos el repositorio local **proyecto-control-tareas** con el nuevo proyecto del grupo.

Primero, revisa los remotos actuales.

```
cd ~/workspace/proyecto-control-tareas
git remote -v
```

Si ya existe un remoto **origin** apuntando al proyecto anterior (fuera del grupo), actualiza la URL.

```
# Opción A: cambiar la URL del remoto existente
git remote set-url origin git@gitlab.ejemplo.com:empresa-formacion/control-tareas-equipo.git

# Opción B: Añadir uno nuevo con otro nombre
git remote add control git@gitlab.com:empresa-formacion/control-tareas-equipo.git

# Verificar el cambio
git remote -v
```

Sube todo el contenido al nuevo proyecto.

```
# Si has cambiado el remoto origin
git push -u origin main
```



```
# Si has añadido un nuevo remoto llamado control
git push -u control main
git push --tags
```

Verifica en el navegador que el proyecto muestra el código, los commits y las tags.

10.3.5. Configurar la protección de ramas

Protege la rama `main` para que los cambios solo puedan integrarse mediante Merge Requests.

1. En el proyecto, ve a **Settings > Repository**.
2. Expande la sección **Protected branches**.
3. Verifica que `main` aparece como rama protegida. Si no aparece, añádela.
4. Configura los permisos:

Permiso	Configuración
Allowed to merge	Maintainers (solo los maintainers pueden hacer merge)
Allowed to push and merge	No one (nadie puede hacer push directo)
Allowed to force push	Desactivado

5. Guarda los cambios.

Comprueba que la protección funciona intentando hacer push directo a `main`.

```
cd ~/workspace/proyecto-control-tareas
echo "# Test de protección" >> README.md
git add README.md
git commit -m "gitlab-lab: test de protección de rama"
git push control main
```

El push debe ser **rechazado** con un mensaje similar a:

```
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to 'gitlab.com:empresa-formacion/control-tareas-equipo.git'
```

Deshaz el commit de prueba.

```
git reset --hard HEAD~1
```

10.3.6. Añadir un miembro al grupo

Invita a un compañero (o crea un segundo usuario de prueba) como miembro del grupo.

1. En el grupo **empresa-formacion**, ve a **Manage > Members**.
2. Haz clic en **Invite members**.
3. Introduce el nombre de usuario o correo electrónico del compañero.
4. Asígnale el rol **Developer**.
5. Haz clic en **Invite**.

Verifica que el nuevo miembro aparece en la lista de miembros del grupo.

Ahora comprueba la herencia de permisos:

1. Ve al proyecto **control-tareas-equipo > Manage > Members**.
2. El compañero debe aparecer como miembro **heredado** (inherited) con rol **Developer**.

10.3.7. Configurar ajustes adicionales del proyecto

Personaliza la configuración del proyecto.

1. Ve a **Settings > General**.
2. En **Naming, topics, avatar**:
 - Añade una descripción: " ".
 - Añade los topics: **curso**, **git**, **gitlab**.
3. En **Visibility, project features, permissions**:
 - Verifica que las **Merge Requests** están habilitadas.
 - Deshabilita la **Wiki** si no se va a utilizar.
4. Guarda los cambios.

10.4. Resultado esperado

Al terminar el laboratorio, el alumno debe tener:

- Un grupo **empresa-formacion** con visibilidad privada en GitLab.
- Un proyecto **control-tareas-equipo** dentro del grupo, con el repositorio local sincronizado.
- La rama **main** protegida contra push directo.
- Un compañero añadido como Developer con permisos heredados del grupo.
- Issues y Merge Requests habilitados en el proyecto.

Capítulo 11. Lab: Gestión de Issues y Milestones (sprint simulado)

11.1. Objetivos

- Crear labels de grupo con scoped labels para clasificar issues.
- Crear y gestionar issues con títulos descriptivos, labels y asignados.
- Crear un milestone que represente un sprint y asociarle issues.
- Configurar un tablero de issues (Issue Board) con columnas por estado.
- Simular un mini-sprint moviendo issues por el tablero y cerrándolos.

11.2. Prerrequisitos

- Grupo `empresa-formacion` y proyecto `control-tareas-equipo` creados en el laboratorio anterior.
- Al menos un compañero añadido como Developer en el grupo (o un segundo usuario de prueba).

11.3. Ejercicios

11.3.1. Crear labels de grupo

Los labels se crean a nivel de **grupo** para que estén disponibles en todos los proyectos.

1. Accede al grupo `empresa-formacion` en GitLab.
2. Ve a **Manage > Labels**.
3. Haz clic en **New label** y crea los siguientes labels:

Label	Color sugerido	Descripción
<code>tipo::feature</code>	Verde (#428BCA)	Funcionalidad nueva
<code>tipo::bug</code>	Rojo (#D9534F)	Defecto o error
<code>tipo::tarea</code>	Gris (#7F8C8D)	Tarea de mantenimiento
<code>prioridad::alta</code>	Rojo (#E11D48)	Prioridad alta
<code>prioridad::media</code>	Naranja (#F0AD4E)	Prioridad media
<code>prioridad::baja</code>	Azul claro (#69D100)	Prioridad baja
<code>estado::pendiente</code>	Gris claro (#F5F5F5)	Issue pendiente de inicio
<code>estado::en-progreso</code>	Azul (#1F78D1)	Issue en curso
<code>estado::en-revision</code>	Morado (#9B59B6)	Issue en revisión de código

NOTE

Los **scoped labels** (etiquetas con `::`) requieren GitLab **Premium** o **Ultimate**. En GitLab Free, los labels se crean con los mismos nombres pero no tienen el comportamiento de exclusión mutua automática.

11.3.2. Crear un milestone (sprint simulado)

Crea un milestone que represente un sprint de una semana.

1. En el **proyecto** `control-tareas-equipo`, ve a **Plan** > **Milestones**.
2. Haz clic en **New milestone**.
3. Completa los campos:

Campo	Valor
Title	Sprint 1 — Mejoras del proyecto
Description	"Primer sprint del equipo. Objetivo: implementar mejoras básicas en la documentación y la estructura del proyecto."
Start date	Fecha de hoy
Due date	Fecha de hoy + 7 días

4. Haz clic en **Create milestone**.

11.3.3. Crear issues para el sprint

Crea los siguientes issues y asócialos al milestone. Cada issue simula una tarea real de un equipo de desarrollo.

1. En el proyecto, ve a **Plan** > **Work items** y haz clic en **New item**.
2. En el desplegable **Type**, selecciona **Issue** (si no está ya seleccionado).

Crea los siguientes issues (uno por uno):

Issue 1:

Campo	Valor
Title	Crear fichero CONTRIBUTING.md con guía de contribución
Description	"Crear un fichero CONTRIBUTING.md en la raíz del proyecto con las normas de contribución del equipo: convenciones de commits, flujo de ramas y proceso de revisión de código."
Assignee	Tu usuario
Labels	<code>tipo::feature</code> , <code>prioridad::alta</code> , <code>estado::pendiente</code>
Milestone	Sprint 1 — Mejoras del proyecto

Issue 2:

Campo	Valor
Title	Corregir errores tipográficos en README.md
Description	"Revisar el fichero README.md y corregir errores ortográficos y de formato."
Assignee	Tu compañero (o segundo usuario)
Labels	<code>tipo::bug</code> , <code>prioridad::media</code> , <code>estado::pendiente</code>
Milestone	Sprint 1 — Mejoras del proyecto

Issue 3:

Campo	Valor
Title	Añadir fichero .editorconfig para consistencia de formato
Description	"Crear un fichero .editorconfig en la raíz del proyecto para estandarizar la configuración del editor (indentación, charset, fin de línea)."
Assignee	Tu usuario
Labels	<code>tipo::tarea</code> , <code>prioridad::baja</code> , <code>estado::pendiente</code>
Milestone	Sprint 1 — Mejoras del proyecto

Issue 4:

Campo	Valor
Title	Documentar la estructura de directorios del proyecto
Description	"Añadir una sección al README.md que describa la estructura de directorios del proyecto y el propósito de cada carpeta."
Assignee	Tu compañero (o segundo usuario)
Labels	<code>tipo::feature</code> , <code>prioridad::media</code> , <code>estado::pendiente</code>
Milestone	Sprint 1 — Mejoras del proyecto

11.3.4. Configurar el tablero de issues

Crea un tablero Kanban con columnas basadas en los labels de estado.

1. Ve a **Plan > Issue boards**.
2. El tablero por defecto tiene las columnas "Open" y "Closed".
3. Haz clic en **New list** (a la derecha de las columnas existentes).
4. Selecciona **Label**, elige el label `estado::pendiente` y haz clic en **Add to board**.
5. Repite para `estado::en-progreso` y `estado::en-revision`.

El tablero debe tener ahora estas columnas:

Open | estado::pendiente | estado::en-progreso | estado::en-revision | Closed

Los 4 issues deberían aparecer en la columna `estado::pendiente` (ya que se crearon con ese label).

11.3.5. Simular el flujo de trabajo del sprint

Ahora simula el avance del sprint moviendo issues por el tablero.

Comenzar a trabajar en el Issue 1

1. En el tablero, arrastra el Issue 1 ("Crear fichero CONTRIBUTING.md") de `estado::pendiente` a `estado::en-progreso`.
2. Observa que el label `estado::pendiente` se ha reemplazado automáticamente por `estado::en-progreso`.

Ahora pasa a la terminal y crea la rama y el contenido para este issue.

```
cd ~/workspace/proyecto-control-tareas
git switch -c feature/1-contributing-md
```

Crea el fichero CONTRIBUTING.md.

```
cat > CONTRIBUTING.md << 'EOF'
# Guía de contribución

## Convenciones de commits

Usamos el estándar Conventional Commits:

- `feat:` para funcionalidades nuevas
- `fix:` para correcciones de errores
- `docs:` para cambios en documentación
- `chore:` para tareas de mantenimiento

## Flujo de ramas

1. Crear una rama desde `main` con el prefijo adecuado: `feature/`, `bugfix/`, `hotfix/`.
2. Desarrollar y hacer commits regulares.
3. Abrir una Merge Request hacia `main`.
4. Solicitar revisión de código a un compañero.
5. Una vez aprobada, hacer merge.

## Proceso de revisión

- Toda Merge Request requiere al menos una aprobación.
- El revisor debe comprobar: funcionalidad, estilo de código y documentación.
- Usar comentarios constructivos y sugerencias aplicables.
```

```
EOF
```

Haz commit y push.

```
git add CONTRIBUTING.md
git commit -m "feat: crear guía de contribución CONTRIBUTING.md

Closes #1"
git push -u control feature/1-contributing-md
```

Completar el Issue 3 (tarea rápida)

Arrastra el Issue 3 ("Añadir .editorconfig") de **estado::pendiente** a **estado::en-progreso** en el tablero.

```
git switch main
git switch -c feature/3-editorconfig
```

```
cat > .editorconfig << 'EOF'
# EditorConfig - https://editorconfig.org
root = true

[*]
indent_style = space
indent_size = 4
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true

[*.md]
trim_trailing_whitespace = false

[Makefile]
indent_style = tab
EOF
```

```
git add .editorconfig
git commit -m "chore: añadir fichero .editorconfig

Closes #3"
git push -u control feature/3-editorconfig
```

Fusionar las ramas y cerrar issues

Para simular la finalización del sprint, fusiona las ramas en **main**. En un flujo real esto se haría mediante Merge Requests (que se practican en el Lab 12), pero aquí completamos el ciclo para ver

el cierre automático de issues.

Dado que **main** está protegida contra push directo, desprotege temporalmente la rama para este ejercicio (o usa la interfaz de GitLab para hacer merge).

En la interfaz de GitLab:

1. Ve al proyecto > **Code** > **Merge requests** y haz clic en **New merge request**.
2. Selecciona **feature/1-contributing-md** como **Source branch** y **main** como **Target branch**. Haz clic en **Compare branches and continue**.
3. Completa el título y la descripción. Asegúrate de que la descripción contiene **Closes #1**.
4. En el apartado Milestone, selecciona "Sprint 1 — Mejoras del proyecto".
5. Selecciona **Create merge request** y, a continuación, haz clic en **Merge** directamente.
6. Repite para **feature/3-editorconfig** con **Closes #3**.

Sincroniza el repositorio local.

```
git switch main
git pull control main
git logall
```

Verificar el estado del sprint

1. Ve al tablero de issues (**Plan** > **Issue boards**).
2. Los Issues 1 y 3 deben estar en la columna **Closed**.
3. Los Issues 2 y 4 deben seguir en **estado::pendiente**.
4. Ve a **Plan** > **Milestones** y abre el Sprint 1.
5. El progreso debe mostrar **50%** (2 de 4 issues cerrados).

11.4. Resultado esperado

Al terminar el laboratorio, el alumno debe poder:

- Crear y utilizar scoped labels para clasificar issues de forma mutuamente excluyente.
- Crear un milestone con fechas y asociarle issues.
- Configurar un tablero Kanban con columnas basadas en labels de estado.
- Mover issues por el tablero y entender cómo se actualizan los labels automáticamente.
- Cerrar issues automáticamente mediante la palabra clave **Closes #X** en commits y merge requests.
- Verificar el progreso del milestone a medida que se completan los issues.

Capítulo 12. Lab: Flujo completo de Merge Request con code review en pareja

12.1. Objetivos

- Crear una feature branch, desarrollar una funcionalidad y publicarla.
- Abrir una Merge Request en GitLab con título, descripción y referencia a un issue.
- Realizar un code review en pareja: comentarios en línea y sugerencias aplicables.
- Aprobar la Merge Request tras la revisión.
- Fusionar la MR y verificar el cierre automático del issue asociado.
- Practicar el flujo completo desde la planificación hasta la integración.

12.2. Prerrequisitos

- Laboratorio anterior completado: grupo, proyecto, labels, milestone e issues existentes.
- Issues 2 y 4 del Sprint 1 aún abiertos (del laboratorio anterior).
- Al menos un compañero con rol Developer en el grupo (para hacer code review cruzado).
- Si no hay compañero disponible, se puede simular el code review con un segundo usuario o haciendo el propio usuario de revisor.

12.3. Ejercicios

12.3.1. Verificar el punto de partida

```
cd ~/workspace/proyecto-control-tareas
git switch main
git pull control main
git status
git logall
```

El repositorio debe estar limpio y en la rama `main`, con los issues 2 y 4 aún abiertos en GitLab.

12.3.2. Desarrollador A: crear rama y trabajar en el Issue 2

El Issue 2 es "Corregir errores tipográficos en README.md". El Desarrollador A (tú) asume este issue.

Primero, mueve el Issue 2 a `estado::en-progreso` en el tablero de issues de GitLab.

Crea una rama vinculada al issue.

```
git switch -c bugfix/2-corregir-readme
```

Modifica el README.md para mejorar su contenido.

```
cat > README.md << 'EOF'
# Proyecto de control de tareas – Equipo Unificado

## Descripción

Aplicación de gestión y seguimiento de tareas para equipos de desarrollo.
Este proyecto se utiliza como base para el curso de GitLab Básico.

## Instalación

1. Clonar el repositorio.
2. Revisar la documentación en la carpeta `docs/`.

## Contribución

Consulta el fichero [CONTRIBUTING.md](CONTRIBUTING.md) para las normas de contribución
del equipo.

## Licencia

Proyecto de formación – uso interno.
EOF
```

Haz commit y push de la rama.

```
git add README.md
git commit -m "fix: corregir y mejorar contenido de README.md

- Añadir sección de descripción
- Añadir sección de instalación
- Añadir referencia a CONTRIBUTING.md
- Añadir sección de licencia

Closes #<numero del issue>"
git push -u control bugfix/2-corregir-readme
```

12.3.3. Desarrollador A: abrir la Merge Request

1. Ve al proyecto en GitLab. GitLab mostrará un banner sugiriendo crear una MR para la rama recién publicada. Haz clic en **Create merge request**.

Si no aparece el banner, ve a **Code > Merge requests > New merge request** y selecciona:
Source branch: `bugfix/2-corregir-readme` **Target branch:** `main`

2. Completa la MR:

Campo	Valor
Title	Corregir y mejorar README.md
Description	<pre>## Cambios realizados - Reorganizado el contenido del README.md - Añadidas secciones: Descripción, Instalación, Contribución y Licencia - Corregidos errores de formato ## Issue relacionado Closes #2</pre>
Assignee	Tu usuario
Reviewer	Tu compañero
Labels	tipo::bug, estado::en-revision
Milestone	Sprint 1 — Mejoras del proyecto

3. Haz clic en **Create merge request**.

12.3.4. Revisor: realizar el code review

El compañero (Revisor) abre la MR y revisa los cambios.

1. El Revisor accede al proyecto y abre la MR desde **Code > Merge requests**.
2. Abre la pestaña **Changes** para ver el diff.

Añadir un comentario en línea

1. Pasa el ratón por la línea que dice **Proyecto de formación – uso interno**. en el diff.
2. Haz clic en el icono de comentario que aparece a la izquierda de la línea.
3. Escribe un comentario:

Sería mejor indicar el tipo de licencia específico o enlazar a un fichero LICENSE.

4. Haz clic en **Start a review** (no en "Add comment now", para agrupar todos los comentarios en una sola revisión).

Hacer una sugerencia aplicable

1. En la línea que dice **2. Revisar la documentación en la carpeta `docs/`**, haz clic en el icono de comentario.
2. Haz clic en el botón de **Insert suggestion** (en el editor icono de página de código al lado de Preview) para insertar un bloque de sugerencia.

3. Modifica el texto sugerido:

```
```suggestion
2. Revisar la documentación en la carpeta `docs/`.
3. Seguir las instrucciones de configuración del entorno.
```
```

4. Añade un texto explicativo sobre la sugerencia: "Propongo añadir un paso más de instalación."

5. Haz clic en **Add to review**.

Enviar la revisión

1. Haz clic en **Your Review**.

2. Selecciona el tipo de revisión:

- **Comment**: comentarios sin aprobación ni rechazo.
- **Approve**: aprobar la MR.
- **Request changes**: solicitar cambios antes de aprobar.

3. Selecciona **Request changes** para este ejercicio.

4. Haz clic en **Submit review**.

12.3.5. Desarrollador A: responder a la revisión

El Desarrollador A (tú) revisa los comentarios del Revisor.

Aplicar la sugerencia

1. Abre la MR y ve a la pestaña **Overview** o **Changes**.

2. Localiza la sugerencia aplicable del Revisor.

3. Haz clic en **Apply suggestion**.

4. GitLab crea un commit automáticamente con el cambio sugerido. Acepta el mensaje de commit por defecto.

Responder al comentario sobre la licencia

1. Localiza el comentario sobre la licencia.

2. Responde al hilo:

```
Buen punto. Lo dejo como tarea pendiente para un issue futuro.
```

3. Haz clic en **Add comment now** y marca el hilo como resuelto.

Actualizar la rama local

La sugerencia aplicada creó un commit en el remoto. Actualiza la rama local.

```
cd ~/workspace/proyecto-control-tareas
git switch bugfix/2-corregir-readme
git pull control bugfix/2-corregir-readme
git logall
```

12.3.6. Revisor: aprobar la Merge Request

El Revisor revisa los cambios adicionales y aprueba la MR.

1. Abre la MR en GitLab.
2. Verifica que todos los hilos de discusión están resueltos.
3. Haz clic en **Approve**.

La MR debe mostrar ahora el estado **Approved** con el nombre del Revisor.

12.3.7. Hacer merge de la MR

Una vez aprobada, el Desarrollador A (o el Maintainer) fusiona la MR.

1. En la MR, localiza la sección de merge en la parte inferior.
2. Selecciona la estrategia de merge:
 - **Merge commit** (por defecto): crea un commit de merge con los dos padres registrados en el historial.
 - Activa la casilla "**Delete source branch**" para eliminar la rama automáticamente.
3. Haz clic en **Merge**.

Verifica en GitLab:

1. Ve a **Plan > Issues**: el Issue 2 debe estar cerrado.
2. Ve a **Plan > Milestones**: el Sprint 1 debe mostrar 3 issues cerrados de 4.
3. Ve a **Code > Branches**: la rama `bugfix/2-corregir-readme` ya no debe existir.

Sincroniza el repositorio local.

```
cd ~/workspace/proyecto-control-tareas
git switch main
git pull control main

# Eliminar la rama local que ya no existe en el remoto
git branch -d bugfix/2-corregir-readme

git logall
```

12.3.8. Desarrollador B: completar el Issue 4 (flujo completo)

Ahora el compañero (Desarrollador B) repite el flujo completo para el Issue 4: "Documentar la estructura de directorios del proyecto".

Si trabajas solo, simula al Desarrollador B en el directorio del Colaborador B. Recuerda configurar el remoto "control" apuntando al repositorio del proyecto para poder sincronizar los cambios. En este caso solo mantendremos el remoto "control" para evitar confusiones con el remoto "origin" del fork.

```
cd ~/workspace/colaborador-b
git remote set-url origin <url-del-repositorio-del-control>
git remote rename origin control
git pull control main
git switch -c feature/4-documentar-estructura
```

Añade la sección de estructura al README.md.

```
cat >> README.md << 'EOF'

## Estructura del proyecto

'''
proyecto-control-tareas/
|—— docs/                # Documentación del proyecto
|   |—— arquitectura.md   # Documento de arquitectura
|   |—— guia-despliegue.md # Guía de despliegue
|—— src/                  # Código fuente (futuro)
|—— .editorconfig         # Configuración del editor
|—— CONTRIBUTING.md       # Guía de contribución
|—— README.md             # Este fichero
'''
EOF
```

```
git add README.md
git commit -m "docs: documentar estructura de directorios del proyecto"

Closes #4"
git push -u control feature/4-documentar-estructura
```

El Desarrollador B abre la MR en GitLab:

1. Ve al proyecto > **Code** > **Merge requests** > **New merge request** > **Compare branches and continue**.
2. Source: **feature/4-documentar-estructura**, Target: **main**.
3. Title: "Documentar estructura de directorios".
4. Description: incluir **Closes #4**.

5. Reviewer: el Desarrollador A.
6. Label: `tipo::feature`, `estado::en-revision`.
7. Milestone: Sprint 1.

Revisar y aprobar la MR

1. El Desarrollador A abre la MR `feature/4-documentar-estructura` en GitLab desde **Code > Merge requests**.
2. Revisa los cambios en la pestaña **Changes** y, si todo es correcto, hace clic en **Approve**.

Hacer merge del issue restante

1. En la sección de merge de la MR, localiza el desplegable de estrategia.
2. Activa "**Delete source branch**".
3. Haz clic en **Merge**.

Sincroniza el directorio del Desarrollador A.

```
cd ~/workspace/proyecto-control-tareas
git switch main
git pull control main
git logall
```

12.3.9. Cerrar el milestone y verificar el resultado final

1. En GitLab, ve a **Plan > Milestones**.
2. El Sprint 1 debe mostrar **100%** de progreso (4 de 4 issues cerrados).
3. Haz clic en **Close milestone**.

Sincroniza ambos directorios locales.

```
cd ~/workspace/colaborador-b
git switch main
git pull control main
git branch -d feature/4-documentar-estructura
git fetch --prune control
git logall
```

12.4. Resultado esperado

Al terminar el laboratorio, el alumno debe poder:

- Abrir una Merge Request con título descriptivo, descripción con **Closes #X** y revisores asignados.
- Realizar un code review: comentar en línea, hacer sugerencias aplicables y solicitar cambios.
- Aplicar sugerencias del revisor directamente desde la interfaz de GitLab.

- Resolver hilos de discusión y marcar la revisión como completada.
- Aprobar y fusionar una MR, verificando el cierre automático del issue y la actualización del milestone.
- Ejecutar el flujo completo: issue → rama → commits → push → MR → review → merge → cierre.

Capítulo 13. Lab: GitLab Duo — extensión VS Code y asistencia IA integrada

13.1. Objetivos

- Activar una versión de prueba de GitLab Duo en la instancia o grupo.
- Crear un **token de acceso personal** (PAT) con scope `api` para autenticar la extensión de VS Code.
- Instalar y configurar la extensión **GitLab for VS Code** (`GitLab.gitlab-workflow`).
- Crear un script bash en el proyecto para usar como base de los ejercicios con IA.
- Explorar las principales funcionalidades de la extensión.
- Practicar los slash commands de Duo Chat.
- Usar GitLab Duo desde VS Code para crear una rama, generar un script, hacer commit y push.
- Explorar las funcionalidades de GitLab Duo desde la **UI web** de GitLab.
- Crear y gestionar una **Merge Request** con asistencia de GitLab Duo.
- Conocer el **GitLab Duo Agent Platform**: foundational agents, custom agents y flujos automatizados.
- Resolver un **conflicto de merge** con ayuda de GitLab Duo en VS Code.

13.2. Prerrequisitos

- Laboratorio anterior completado: proyecto `control-tareas-equipo` con historial, MR fusionadas y milestone cerrado.
- Acceso a una instancia GitLab con licencia **Premium** o **Ultimate**.
- VS Code instalado.
- Repositorio local `proyecto-control-tareas` actualizado y en la rama `main`, con el remoto `control` configurado.

13.3. Ejercicios

13.3.1. Activar la versión de prueba de GitLab Duo

Antes de usar las funcionalidades de IA de GitLab Duo, es necesario activar el add-on en la instancia o grupo.

En **GitLab.com**

1. Accede a <https://customers.gitlab.com/> con la cuenta del grupo.
2. En la tarjeta de suscripción, selecciona los tres puntos verticales.
3. Selecciona **Buy GitLab Duo Pro** (o solicita una prueba desde <https://about.gitlab.com/solutions/gitlab-duo-pro/sales/>).

4. Indica el número de asientos necesarios y completa la compra.

En GitLab Self-Managed

1. El administrador debe acceder a **Admin > Subscription** y sincronizar la suscripción.
2. Ve a **Admin > GitLab Duo**.
3. En **Seat utilization**, selecciona **Assign seats**.
4. Activa el toggle a la derecha del usuario que necesite acceso.

Verificar la activación

1. En la barra superior de GitLab, ve a tu grupo.
2. Selecciona **Settings > GitLab Duo**.
3. Confirma que el usuario aparece con asiento asignado y que las funcionalidades de Duo están habilitadas.

13.3.2. Crear un token de acceso personal con scope api

La extensión de GitLab para VS Code necesita un **Personal Access Token** (PAT) con el scope **api** para autenticarse contra la instancia de GitLab.

1. En GitLab, pulsa tu avatar en la esquina superior derecha y selecciona **Edit profile**.
2. En el menú lateral izquierdo, selecciona **Access Tokens**.
3. Pulsa **Add new token**.
4. Configura el token:
 - **Token name:** **vscode-gitlab-duo**
 - **Expiration date:** establece una fecha (por ejemplo, 90 días).
 - **Scopes:** marca **api**.
5. Pulsa **Create personal access token**.
6. **Copia el token** inmediatamente y guárdalo en un lugar seguro. No se podrá volver a ver.

IMPORTANT

No compartas este token ni lo subas a ningún repositorio. Trátalo como una contraseña.

13.3.3. Instalar la extensión GitLab for VS Code

1. Abre VS Code.
2. Ve a la vista de **Extensions** (**Ctrl+Shift+X**).
3. Busca **GitLab** en el marketplace.
4. Instala la extensión **GitLab** del editor **GitLab** (identificador: **gitlab.gitlab-workflow**).
5. Una vez instalada, aparecerá el icono de GitLab (tanuki) en la barra lateral izquierda.

Autenticarse con el PAT

1. Abre la **Command Palette** (**Ctrl+Shift+P**).
2. Escribe **GitLab: Authenticate** y pulsa Enter.
3. Selecciona la URL de tu instancia de GitLab o introdúcela manualmente (por ejemplo, <https://gitlab.com> o la URL de tu instancia self-managed).
4. Selecciona **PAT** como método de autenticación.
5. Pega el token creado en el paso anterior.

Conectar con el repositorio

Asegúrate de tener el proyecto abierto en VS Code:

```
cd ~/workspace/proyecto-control-tareas
code .
```

Verifica que la extensión detecta el proyecto:

1. En la barra lateral izquierda, pulsa el icono de **GitLab** (tanuki).
2. Debería aparecer el proyecto **control-tareas-equipo** con sus issues y merge requests.
3. Si tienes múltiples remotos, selecciona el proyecto correcto pulsando en la línea del proyecto.

Verifica la barra de estado de VS Code: si hay una pipeline para el último commit o una MR asociada a la rama actual, la extensión mostrará esa información en la parte inferior.

13.3.4. Crear un script base para los ejercicios con IA

Antes de explorar Duo Chat, necesitas un fichero de código real en el proyecto sobre el que practicar los slash commands. Crea un script bash que genera un resumen del estado del repositorio.

Desde el terminal integrado de VS Code (**Ctrl+ñ** o **Ctrl+backtick**):

```
cd ~/workspace/proyecto-control-tareas
git switch main
git pull control main
```

Crea una rama nueva para añadir el script:

```
git switch -c feat/script-resumen
```

Crea el directorio y el script:

```
mkdir -p scripts
cat > scripts/resumen.sh << 'EOF'
```

```
#!/usr/bin/env bash
# resumen.sh – Genera un resumen rápido del estado del repositorio.
# Uso: bash scripts/resumen.sh

set -euo pipefail

echo "=====
echo "  RESUMEN DEL PROYECTO"
echo "=====
echo ""
echo "Rama actual:      $(git branch --show-current)"
echo "Último commit:    $(git log -1 --format='%h %s')"
echo "Total commits:     $(git rev-list --count HEAD)"
echo "Ramas locales:     $(git branch | wc -l | tr -d ' ')"
echo "Ramas remotas:     $(git branch -r | wc -l | tr -d ' ')"
echo "Tags:              $(git tag | wc -l | tr -d ' ')"
echo ""

echo "--- Ficheros por tipo ---"
echo "  .sh:  $(find . -name '*.sh' | wc -l | tr -d ' ')"
echo "  .md:  $(find . -name '*.md' | wc -l | tr -d ' ')"
echo "  .txt: $(find . -name '*.txt' | wc -l | tr -d ' ')"
echo ""

echo "--- Últimos 5 commits ---"
git log -5 --format=' %h %s (%cr)' --date=relative
echo ""
echo "=====
EOF
chmod +x scripts/resumen.sh
```

Confirma que funciona (opcional, no es el objetivo del ejercicio):

```
bash scripts/resumen.sh
```

Haz commit del script y publica la rama:

```
git add scripts/resumen.sh
git commit -m "feat: añadir script de resumen del repositorio"
git push control feat/script-resumen
```

Fusionar la rama mediante Merge Request

1. En GitLab, abre el chat de GitLab Duo y pide que genere un MR para fusionar **feat/script-resumen** en **main**
2. En visual Studio Code comprueba, en el icono de GitLab en la barra lateral, que la MR ha sido creada. Debería aparecer en la sección de Merge Requests.
3. Revisa los cambios y pulsa **Merge**.

4. Marca la opción de eliminar la rama de origen si está disponible.

Actualiza el repositorio local:

```
git switch main
git pull control main
git branch -d feat/script-resumen
```

13.3.5. GitLab Duo Chat en VS Code

GitLab Duo Chat es un asistente de IA integrado en el IDE que permite hacer preguntas sobre código, el proyecto y GitLab en general.

1. Abre **Duo Chat** desde la barra lateral de GitLab o con la **Command Palette**: **GitLab: Open Duo Chat**.
2. Prueba los siguientes prompts:

Preguntar sobre GitLab:

```
¿Cómo puedo crear una Merge Request desde la línea de comandos?
```

Explicar código (**/explain**):

1. Abre el fichero **scripts/resumen.sh** en VS Code.
2. Selecciona todo el contenido del script.
3. En Duo Chat, escribe **/explain**.
4. Revisa la explicación generada: ¿identifica correctamente la función de cada bloque?

Refactorizar código (**/refactor**):

1. Con **scripts/resumen.sh** seleccionado, escribe en Duo Chat:

```
/refactor mejorar la portabilidad POSIX y excluir el directorio .git de los find
```

2. Revisa las sugerencias. Duo debería proponer:
 - Sustituir **echo** por **printf** para mayor portabilidad.
 - Añadir **-not -path './.git/*'** a los **find**.
 - Validar que se ejecuta dentro de un repositorio Git.
3. Si alguna sugerencia te convence, aplícala manualmente al script.

Corregir código (**/fix**):

1. Selecciona el bloque de **find** del script y escribe en Duo Chat:

```
/fix los find recorren el directorio .git y cuentan ficheros internos de Git
```

2. Compara la corrección propuesta con la refactorización anterior: ¿son coherentes?

Generar tests (/tests):

1. Selecciona todo el contenido de `scripts/resumen.sh`.
2. Escribe en Duo Chat:

```
/tests usando bats-core
```

3. Examina los tests generados. Analiza:

- ¿Cubren los casos principales (ejecución correcta, ejecución fuera de un repo)?
- ¿Usan mocks para los comandos Git o ejecutan Git real?
- No es necesario ejecutarlos; el objetivo es evaluar la calidad de la propuesta.

Incluir ficheros en contexto (/include):

1. En Duo Chat, escribe `/include` y selecciona `scripts/resumen.sh` y `README.md`.
2. Haz una pregunta que relacione ambos ficheros:

```
¿Debería documentar el script resumen.sh en el README? Si es así, genera la sección Markdown correspondiente.
```

13.3.6. Crear un script con ayuda de GitLab Duo y publicarlo

En este ejercicio usarás GitLab Duo para asistirte en la creación de un script, gestión de rama, commit y push.

Crear una rama nueva

Desde VS Code, abre el terminal integrado (`Ctrl+ñ` o `Ctrl+backtick`):

```
cd ~/workspace/proyecto-control-tareas
git switch main
git pull control main
git switch -c feat/monitor-estado
```

Generar el script con Duo Chat

Abre **Duo Chat** y pide que genere un script de monitorización:

```
Genera un script bash llamado scripts/monitor.sh que:
1. Compruebe si hay cambios sin confirmar en el repositorio.
```

2. Muestre las ramas locales que no tienen equivalente en el remoto.
3. Liste los últimos 3 tags creados.
4. Muestre si hay stashes pendientes.

Usa `set -euo pipefail` y añada comentarios explicativos.

Revisa el código generado por Duo. Cópialo y crea el fichero:

```
mkdir -p scripts
# Pega el contenido generado por Duo en el fichero:
cat > scripts/monitor.sh << 'EOF'
#!/usr/bin/env bash
# monitor.sh – Monitoriza el estado del repositorio Git.
# Generado con asistencia de GitLab Duo.

set -euo pipefail

echo "=== Estado del repositorio ==="
echo ""

# Cambios sin confirmar
if [[ -n $(git status --porcelain) ]]; then
    echo "[!] Hay cambios sin confirmar:"
    git status --short
else
    echo "[OK] No hay cambios pendientes."
fi
echo ""

# Ramas locales sin equivalente en el remoto
echo "--- Ramas locales sin remoto ---"
git branch --format='%(refname:short)' | while read -r branch; do
    if ! git config "branch.${branch}.remote" > /dev/null 2>&1; then
        echo "  $branch (sin remoto configurado)"
    fi
done
echo ""

# Últimos 3 tags
echo "--- Últimos 3 tags ---"
git tag --sort=-creatordate | head -3
echo ""

# Stashes pendientes
stash_count=$(git stash list | wc -l | tr -d ' ')
if [[ "$stash_count" -gt 0 ]]; then
    echo "[!] Hay $stash_count stash(es) pendientes:"
    git stash list
else
    echo "[OK] No hay stashes pendientes."
fi
```

```
echo ""
echo "=== Fin del informe ==="
EOF
chmod +x scripts/monitor.sh
```

Hacer commit y push

Usa GitLab Duo Code Suggestions para generar el mensaje de commit. Desde la vista de **Source Control** (**Ctrl+Shift+G**):

1. Añade el fichero al staging: pulsa el icono **+** junto a **scripts/monitor.sh**.
2. Si pulsamos el icono de estrella (sparkle) junto al campo de mensaje de commit y estamos logueados en GitHub y disponemos de una cuenta con Copilot podremos generar un mensaje de commit automáticamente.
3. En el chat de GitLab Duo, escribe:

```
Genera un mensaje de commit para añadir el script de monitorización del repositorio.
Sigue el formato Conventional Commits.
```

1. Revisa el mensaje generado. Debería seguir un formato similar a:

```
feat: añadir script de monitorización del repositorio
```

2. Si es correcto, acepta y confirma el commit.

Alternativamente, desde el terminal:

```
git add scripts/monitor.sh
git commit -m "feat: añadir script de monitorización del repositorio"
```

Publica la rama:

```
git push control feat/monitor-estado
```

13.3.7. Explorar GitLab Duo desde la UI web de GitLab

GitLab Duo también ofrece funcionalidades directamente desde la interfaz web de GitLab, sin necesidad de usar un IDE.

Duo Chat en la UI web

1. En GitLab, pulsa el icono de **GitLab Duo Chat** en la esquina inferior derecha (o en la barra lateral).
2. Prueba los siguientes prompts:

Preguntar sobre el proyecto:

Genera un resumen del proyecto control-tareas-equipo.

Preguntar sobre una issue:

Resume la issue #1

Preguntar sobre un commit:

Explica los cambios del último commit de la rama feat/monitor-estado

Explicar código desde la UI

1. Navega a **Code** > **Repository** en el proyecto.
2. Abre el fichero `scripts/monitor.sh`.
3. Selecciona un bloque de código.
4. Pulsa el botón de **Explain code** que aparece sobre la selección (o usa Duo Chat con el enlace al fichero).
5. Revisa la explicación generada.

13.3.8. Crear una Merge Request y gestionarla con GitLab Duo

Crear la MR

1. Abre el chat de GitLab Duo desde la UI web.
2. Pide que genere una Merge Request para fusionar la rama `feat/monitor-estado` en `main`, con el título `feat: añadir script de monitorización del repositorio` y una descripción que explique qué hace el script `monitor.sh`.
3. Revisa que la MR ha sido creada correctamente en GitLab y que la descripción es correcta.
4. Añade un revisor a la MR, si no se puede realizar esta acción con Duo Chat, hazlo manualmente desde la UI.

Usar Duo Chat para revisar la MR

Con la MR abierta, abre **Duo Chat** y prueba:

¿Qué ficheros se han modificado en esta MR?

¿Cómo se puede mejorar esta Merge Request?

1. Añade un comentario en la MR con alguna sugerencia de mejora basada en las

recomendaciones de Duo Chat y pregúntale:

¿Qué opinan los revisores sobre esta MR?

Fusionar la MR

1. Revisa que todo sea correcto.
2. Pulsa **Merge**.
3. Marca la opción de eliminar la rama de origen si está disponible.

Actualiza el repositorio local:

```
git switch main
git pull control main
git branch -d feat/monitor-estado
```

13.3.9. GitLab Duo Agent Platform

El **GitLab Duo Agent Platform** es la plataforma de agentes IA de GitLab. Integra asistentes inteligentes especializados a lo largo de todo el ciclo de desarrollo de software. En lugar de seguir un flujo lineal, permite delegar tareas rutinarias a agentes de IA especializados.

Referencia oficial: https://docs.gitlab.com/user/duo_agent_platform/

Componentes del Agent Platform

El Agent Platform se compone de tres tipos de elementos:

| Componente | Descripción | Licencia mínima |
|----------------------------|---|-------------------------------|
| Foundational Agents | Agentes preconfigurados por GitLab para tareas comunes: planificación, seguridad, análisis de datos, CI/CD. | Premium |
| Foundational Flows | Flujos automatizados que combinan múltiples agentes para resolver tareas del SDLC: generar MRs desde issues, reparar pipelines, revisar código. | Premium |
| Custom Agents | Agentes creados por el usuario con system prompts y herramientas específicas para necesidades del equipo. | Free (con créditos) / Premium |

Foundational Agents

Los **Foundational Agents** son agentes especializados mantenidos por GitLab:

| Agente | Especialidad |
|-------------------------|--|
| Planner | Planificación de producto: crear y priorizar issues, organizar epics y milestones. |
| CI Expert | Crear, depurar y optimizar pipelines de GitLab CI/CD. |
| Security Analyst | Análisis de seguridad: triaje de vulnerabilidades, explicación de hallazgos, generación de correcciones. |
| Data Analyst | Análisis de métricas del proyecto y visualización de datos de desarrollo. |

Estos agentes están activados por defecto y se pueden usar directamente desde Duo Chat.

Probar el Planner Agent:

1. En VS Code, abre **GitLab Duo Agent Platform** en la barra lateral izquierda (icono de chat agéntico).
2. Selecciona la pestaña **Chat**.
3. En el desplegable de **New chat**, selecciona **Planner**.
4. Escribe:

Crea 3 issues para el proyecto control-tareas-equipo:

1. Añadir validación de argumentos al script resumen.sh
2. Crear documentación de uso de los scripts
3. Añadir tests automatizados con bats-core

5. Observa cómo el Planner descompone la petición y crea las issues.
6. Ve a GitLab > **Issues** y verifica que se han creado correctamente.

Custom Agents: crear un agente personalizado

Los **Custom Agents** permiten crear agentes IA adaptados a las necesidades específicas del equipo. Se configuran con un **system prompt**, herramientas autorizadas y se pueden compartir dentro del grupo.

Crear un agente revisor de scripts:

1. En GitLab, ve al proyecto **control-tareas-equipo**.
2. Selecciona **Automate > Agents**.
3. Pulsa **New agent**.
4. Configura el agente:
 - **Display name:** **Revisor de Scripts**
 - **Description:** **Revisa scripts bash del proyecto buscando errores, mejoras de seguridad y portabilidad POSIX.**
 - **Visibility:** **Private** (solo visible para miembros del proyecto).

- **System prompt:**

Eres un revisor experto en administración de sistemas y shell scripting.

Cuando el usuario te pida revisar código:

1. Identifica errores potenciales y edge cases.
2. Evalúa la seguridad: inyección de comandos, variables sin comillas, uso de eval.
3. Propón mejoras de portabilidad POSIX.
4. Verifica que se sigan las buenas prácticas: set -euo pipefail, funciones, comentarios.
5. Sé conciso. Usa formato Markdown con bloques de código para las correcciones.

- **Tools:** selecciona **Read file** para que el agente pueda leer ficheros del repositorio.

5. Pulsa **Create agent**.

Habilitar y usar el agente:

1. En **Automate > Agents**, selecciona la pestaña **Managed**.
2. Selecciona el agente **Revisor de Scripts** y pulsa **Enable**.
3. Selecciona el proyecto **control-tareas-equipo** y confirma.

Desde VS Code:

1. En la barra lateral, abre **GitLab Duo Agent Platform**.
2. Selecciona la pestaña **Chat**.
3. En el desplegable de **New chat**, selecciona **Revisor de Scripts**. (Si no aparece, asegúrate de que el agente está habilitado y asignado al proyecto. Desloguea y vuelve a iniciar sesión si es necesario).
4. Escribe:

Revisa el fichero scripts/resumen.sh y dame un informe de mejoras.

5. Compara las sugerencias del agente personalizado con las que daba Duo Chat genérico con **/refactor**. ¿Son más específicas?

Visión general del Agent Platform

Para cerrar esta sección, revisa en GitLab las opciones disponibles del Agent Platform:

1. Ve a **Explore > AI Catalog** en la barra superior.
2. Selecciona la pestaña **Agents**: verás los foundational agents y los custom agents públicos.
3. Selecciona la pestaña **Flows**: verás los foundational flows disponibles.
4. En el proyecto, comprueba **Automate > Agents** para ver los agentes habilitados.

| Componente | Dónde se usa | Cuándo elegirlo |
|---------------------|------------------|---|
| Duo Chat (genérico) | VS Code / UI web | Preguntas rápidas, explicar código, generar snippets. |
| Foundational Agent | VS Code / UI web | Tareas especializadas: planificación, CI/CD, seguridad. |
| Custom Agent | VS Code / UI web | Flujos específicos del equipo con prompts y herramientas propias. |
| Foundational Flow | UI web | Automatizar tareas completas: issue → MR, reparar pipelines. |

13.3.10. Resolver un conflicto de merge con ayuda de GitLab Duo

En este ejercicio crearás intencionadamente un conflicto de merge y usarás GitLab Duo en VS Code para resolverlo.

Preparar el conflicto

Asegúrate de estar en `main` actualizado:

```
cd ~/workspace/proyecto-control-tareas
git switch main
git pull control main
```

Crea dos ramas que modifiquen la misma línea del mismo fichero:

Rama A — modifica el título del README:

```
git switch -c conflict/readme-titulo-a
# Modifica la primera línea del README.md
sed -i '1s/.*$/# Control de Tareas – Panel de Equipo/' README.md
git add README.md
git commit -m "docs: actualizar título del README (versión A)"
git switch main
```

Rama B — modifica la misma línea con otro texto:

```
git switch -c conflict/readme-titulo-b
sed -i '1s/.*$/# Proyecto Control de Tareas v2/' README.md
git add README.md
git commit -m "docs: actualizar título del README (versión B)"
```

Fusionar la rama A primero

1. En GitLab, crea una MR para `conflict/readme-titulo-a` → `main`.
2. Fusiónala directamente (no debería haber conflictos).

Actualiza **main** en local:

```
git switch main
git merge conflict/readme-titulo-a
```

Intentar fusionar la rama B

Ahora intenta hacer merge de la rama B en **main**. Habrá un conflicto:

```
git merge conflict/readme-titulo-b
```

Git mostrará un mensaje de conflicto en **README.md**.

Resolver el conflicto con GitLab Duo

1. Abre **README.md** en VS Code. Verás los marcadores de conflicto:

```
<<<<<<< HEAD
# Proyecto Control de Tareas v2
=====
# Control de Tareas – Panel de Equipo
>>>>>>> main
```

2. Selecciona el bloque completo del conflicto (incluyendo los marcadores).
3. Abre **Duo Chat** de **Duo agent platform** y pide ayuda:

```
Arregla el error de README.md manteniendo ambas versiones
```

4. Revisa la sugerencia de Duo. Debería proponer algo como:

```
# Control de Tareas – Panel de Equipo v2
```

5. Aplica la resolución (manual o aceptando la sugerencia de Duo).
6. Elimina los marcadores de conflicto si quedan restos.
7. Confirma la resolución:

```
git add README.md
git commit -m "merge: resolver conflicto en título del README"
```

Crear la MR y fusionar

1. En GitLab, crea una MR para **conflict/readme-titulo-b** → **main**.
2. Verifica que no queden conflictos pendientes.

3. Fusiona la MR.

Limpieza local:

```
git branch -d conflict/readme-titulo-a  
git branch -d conflict/readme-titulo-b
```

13.4. Resultado esperado

Al finalizar este laboratorio:

- Has activado GitLab Duo y creado un token de acceso personal para la extensión de VS Code.
- Has instalado y configurado la extensión **GitLab for VS Code**, explorando sus paneles de issues, MRs y pipelines.
- Has usado **Duo Chat** en VS Code con los slash commands `/explain`, `/refactor`, `/tests`, `/fix` e `/include`.
- Has generado un script con asistencia de Duo, creado una rama, hecho commit y push desde VS Code.
- Has explorado las funcionalidades de GitLab Duo desde la UI web: Chat, explicación de código y generación de descripciones de MR.
- Has conocido el **GitLab Duo Agent Platform**: foundational agents, foundational flows y custom agents.
- Has creado un **custom agent** personalizado y comparado sus resultados con Duo Chat genérico.
- Has creado y resuelto un conflicto de merge con ayuda de GitLab Duo en VS Code.